

Production costs and total completion time minimization for three-stage mixed-model assembly job shop scheduling with lot streaming and batch transfer

Lixin Cheng^{a,b}, Qiuhua Tang^{a,b,*}, Liping Zhang^{a,b}

^a Key Laboratory of Metallurgical Equipment and Control Technology of Ministry of Education, Wuhan University of Science and Technology, China

^b Hubei Key Laboratory of Mechanical Transmission and Manufacturing Engineering, Wuhan University of Science and Technology, China

ARTICLE INFO

Keywords:

Mixed-model assembly job shop scheduling
Lot streaming
Batch transfer
Coevolutionary algorithm
Rule
Q-learning

ABSTRACT

Assembly scheduling problems have broad applications in discrete manufacturing enterprises and hence gained extensive attention in academics. However, some important and actual factors including lot streaming, batch transfer of components and mixed-model assembly are yet not well studied. To adapt to actual production and enhance competitiveness, a three-stage mixed-model assembly job shop scheduling with lot streaming and batch transfer is investigated in this work. To tackle this problem, spatial-temporal links among three stages with differentiated production lot size are established and a novel mathematical model is formulated to minimize total completion time and production costs including setup costs, transport costs, and inventory costs. Besides, an improved multi-objective coevolutionary simulated annealing algorithm is developed. To promote the coevolution of processing, transfer, and assembly sub-problems, the multi-attribute combination rule, similarity rule, and consistency rule are respectively designed for each sub-problem. An adaptive execution strategy of sub-problems is trained by Q-learning to maximize the overall benefits. Experimental results show that both the discovered rule and the adaptive strategy are effective, and the proposed algorithm significantly outperforms seven competitive multi-objective algorithms in fixing the studied problem. More importantly, the developed production mode of “processing with lot streaming-batch transfer-mixed-model assembly considering pull production” has a better comprehensive performance than the other three actual production modes.

1. Introduction

In many discrete manufacturing enterprises, numerous components need to be first processed according to a specific processing route and then assembled in the assembly stage to form final products. This production scheduling problem is viewed as job shop scheduling appending an assembly stage and denoted as the assembly job shop scheduling problem in previous studies (Wong and Ngan, 2013). With the increasing demand for personalized customization, multi-variety and small-batch production mode has become mainstream. Mixed-model assembly lines are widely adopted to achieve the collinear assembly of diverse products (Tang et al., 2021). Quite different from those widely studied assembly job shop scheduling cases where all assembly operations are performed on a single machine or parallel machines, mixed-model assembly demands that components of each product are assigned to a set of stations on the

mixed-model assembly lines and assembled on these stations according to flow production. In this scenario, those components finished earlier can be timely assembled on the corresponding stations to reduce inventory costs and improve production efficiency.

During the processing of multi-variety and small-batch components, a great number of setups are required. To reduce setups, identical components are usually grouped into a large batch (called a lot) and processed continuously on the site. In this scenario, those components finished earlier cannot start until all components in a lot are processed, which leads to numerous waiting time and greatly increase work-in-process (WIP) inventory. To make a balance between setup costs and inventory costs, lot streaming technology (LS) is introduced. In this scenario, each lot is split into multiple smaller sublots so that operations of sublots from a lot can be overlapped to greatly reduce waiting time at the cost of a slight increase of setups.

* Corresponding author. Key Laboratory of Metallurgical Equipment and Control Technology of Ministry of Education, Wuhan University of Science and Technology, China.

E-mail address: tangqiuhua@wust.edu.cn (Q. Tang).

<https://doi.org/10.1016/j.engappai.2023.107729>

Received 30 August 2023; Received in revised form 14 November 2023; Accepted 14 December 2023

Available online 22 December 2023

0952-1976/© 2023 Elsevier Ltd. All rights reserved.

In addition, numerous components finished in the processing stage need to be timely transferred to the assembly stage, and the actual transportation time is large and non-ignorable. However, to simplify the production model, transfer activities are usually not considered in most studies (Basir et al., 2018; Cheng et al., 2022; Komaki and Kayvanfar, 2015; Tang and Zhang, 2021), which makes the developed model inconsistent with the actual production or even infeasible. Thus, a transfer stage should be added. Moreover, instead of transferring one component at a time, batch transfer can be introduced to save transport costs. In this scenario, components from different products can be assigned to a batch and transferred together to reduce the number of transfer batches and facilitate the timely transfer.

In sum, to adapt to actual production and enhance competitiveness in the intensive market competition, this work addresses a three-stage mixed-model assembly job shop scheduling considering lot streaming and batch transfer (MAJSP-LS-BT), and two types of objectives: production efficiency-related one and production costs-related one, are simultaneously optimized. The main contributions of this work are listed below.

- (1) A mixed-integer linear programming (MILP) model considering lot streaming constraints, setup constraints, batch transfer constraints, mixed-model assembly constraints, and spatial-temporal links among three stages with differentiated production lot size is formulated for the MAJSP-LS-BT.
- (2) A coevolutionary simulated annealing algorithm equipped with two-stage decoding, rule-based initialization, and adaptive execution strategy of sub-problems is designed to achieve the co-evolution of processing, transfer, and assembly sub-problems.
- (3) Four production scenarios considering different modes of processing, transfer, and assembly are designed and compared to reveal the superiority of the developed production mode of “processing with lot streaming-batch transfer-mixed-model assembly considering pull production.

The remainder of this work is organized below. Section 2 is the literature review. Section 3 formulates a MILP model for the MAJSP-LS-BT. Section 4 provides an effective algorithm to solve the MAJSP-LS-BT. Section 5 conducts a series of experiments. Section 6 draws a conclusion.

2. Literature review

With a wide range of applications in cars, motor industries, moulds, ball valves, and so on (Fernandez-Viagas et al., 2022), assembly scheduling problems (ASP) have received extensive attention in academics. Research on ASP can be classified as two-stage ASP (Basir et al., 2018), three-stage ASP (Lei et al., 2022), and multi-stage ASP (Sheikh et al., 2019). For the two-stage ASP, components are first processed in the processing stage and then joined together in the assembly stage to form products. Generally, the assembly stage is the final stage of the whole production for those products with a single-level assembly structure (Wang and Lu, 2021). Nevertheless, for those products with a multi-level assembly structure, low-level sub-assemblies may need to be further processed and then assembled to form high-level sub-assemblies or products (Guo et al., 2021). For the three-stage ASP, a transfer stage is added for transferring the components finished in the processing stage to the assembly stage. As for the multi-stage ASP, pre-assembly stages (Pang et al., 2021) or other stages such as painting, inspection and packaging (Sheikh et al., 2019) are considered.

Among the above three types of ASP, the two-stage ASP received the most attention, while few works have been done on three-stage or multi-stage ASP. Moreover, in the existing three-stage or multi-stage ASP, nearly all of them focused on assembly flow-shop scheduling where all components have the same processing requirements, while almost no research has been reported on assembly job shop scheduling with each component having a specific processing route. To fill this gap, this work

investigates a three-stage assembly job shop scheduling problem. Moreover, new attributes including lot streaming, batch transfer, and mixed-model assembly are respectively designed for the processing, transfer, and assembly stages to meet actual production.

2.1. Lot streaming scheduling problems

Lot streaming is a widely used technology for improving production efficiency by splitting each production lot into several sublots so that its operations can be overlapped. The splitting strategies include equal sublots, consistent sublots, and variable sublots. In the case of equal sublots, sublots from a lot have the same subplot size. As for the consistent sublots, it demands that the subplot size and subplot quantity remain unchanged throughout the process. While for the variable sublots, both the number and sizes of sublots can be altered if necessary. Although variable sublots supports more splitting schemes, it was too flexible and hard to control when it was applied to real-world production (Beren and Ömer, 2022). On the contrary, consistent sublots especially the equal and consistent sublots, was easy to implement and received a lot of attention in both academics and industries. In addition, sublots can be executed with intermingling or non-intermingling. With the non-intermingling setting, all sublots of a lot must be executed consecutively without interruption. Obviously, constrained by the non-intermingling setting, some better global scheduling schemes may be missed (Novas, 2019). Thus, the equal and consistent sublots with intermingling is considered in this work.

Till now, lot streaming has been widely applied to various production environments, especially the flow-shop environments. For example, Wang et al. (2022), Zhang et al. (2021), Gong et al. (2018) and Chen et al. (2023) respectively applied lot streaming to the flow-shop, hybrid flow-shop, blocking flow-shop, and distributed flow-shop. As for the job shop environments, Xie et al. (2023) studied the lot streaming job shop scheduling problems. Božek and Werner (2018) and Tutumlu and Sarıç (2023) respectively extend the above research by considering flexible processing routines.

As another important extension of lot streaming job shop scheduling problems, lot streaming assembly job shop scheduling problems are more complex. In this regard, Chan et al. (2008) first investigated the assembly job shop scheduling problem with variable sublots and equal sublots. In this work, a genetic algorithm was applied to determine lot split, and simple dispatching rules were utilized to determine the subplot sequence. Later, Wong et al. (2009) extended the above research by considering resource constraints and proposed a genetic algorithm to simultaneously optimize lot split and subplot sequence. Further, Wong and Ngan (2013) investigated the above lot streaming assembly job shop scheduling problem with part sharing in the assembly stage. As for the lot streaming assembly job shop scheduling problems with flexible processing routines, Li et al. (2022) addressed it with unequal and consistent sublots. Zhang et al. (2020) investigated it with part sharing and setup, and a multi-objective distributed ant colony system was developed to optimize makespan, total tardiness, and total workload. Different from the above research where lot streaming was only applied to the processing stage, Daneshamooz et al. (2022) addressed the flexible assembly job shop scheduling with a parallel assembly, in which lot streaming was applied to both the processing and assembly stages. In these works, assembly operations cannot start until all component lots of a product are finished and at least one piece of product can be produced.

2.2. Assembly scheduling problems considering transfer

Transfer is an indispensable activity in the whole production process. To coordinate the scheduling of production and transportation, more and more studies have been done on scheduling problems considering transfer. Regarding the transfer mode, two commonly witnessed modes: single-piece transfer and batch transfer are widely studied. The former transfers one item at a time, while the latter transfers a batch of items at

Table 1
Summary of assembly scheduling problems.

References	Problem description	Processing stage	Transfer stage	Assembly stage	Objectives
Komaki and Kayvanfar (2015)	Assembly flow-shop scheduling	Release time	–	Single machine	Completion time
Fattahi et al. (2020)	Assembly flow-shop scheduling	Flexible process routes	–	Single machine	Makespan
Zhao et al. (2021)	Distributed assembly no-wait flow-shop scheduling	No-wait constraints	–	Single machine	Completion time
Wu et al. (2018)	Distributed assembly flexible job-shop scheduling	Flexible process routes	–	Single machine	Production costs/earliness
Al-Anzi and Allahverdi (2013)	Assembly flow-shop scheduling		–	Identical assembly machines	Completion time
Tang and Zhang (2021)	Assembly flow-shop scheduling		–	Identical assembly machines	Total completion time/maintenance time
Zhang et al. (2021)	Assembly flow-shop scheduling		–	Mixed-model assembly	Maintenance costs/makespan
Basir et al. (2018)	Assembly flow-shop scheduling		–	Identical assembly machines	Weighted tardy jobs/delivery costs
Kazemi et al. (2017)	Assembly flow-shop scheduling		–	Identical assembly machines	Tardiness/delivery costs
Wang et al. (2016)	Assembly flow-shop scheduling		–	Identical assembly machines	Average arrival time/delivery costs
Yang and Xu (2021)	Distributed assembly flow-shop scheduling		–	Parallel machines	Delivery/tardiness costs
Wang et al. (2021)	Assembly job-shop scheduling		–	Parallel assembly groups	Earliness penalty/completion time cost
Ren et al. (2021)	Assembly flexible job-shop scheduling	Flexible process routes	–	Parallel assembly stations	Energy consumption
Cheng et al. (2022)	Assembly job-shop scheduling		–	Mixed-model assembly	Total completion time/inventory time
Chan et al. (2008)	Assembly job-shop scheduling	Lot streaming	–	Single machine	Lateness/inventory costs
Wong et al. (2009)	Assembly job-shop scheduling	Lot streaming	–	Single machine	Total lateness cost
Wong and Ngan (2013)	Assembly job-shop scheduling	Lot streaming	–	Single machine	Makespan
Li et al. (2022)	Flexible assembly job-shop scheduling	Lot streaming, flexible process routes	–	No assembly time	Flow time
Zhang et al. (2020)	Flexible assembly job-shop scheduling	Lot streaming, setup times	–	Identical assembly robots	Makespan/total tardiness/total workload
Daneshmooz et al. (2022)	Flexible assembly job-shop scheduling	Lot streaming	–	Parallel assembly stations	Makespan
Hatami et al. (2010)	Three-stage assembly flow-shop scheduling	Setup times	Components of one product are transferred together	Single machine	Flow time/tardiness
Shoaardebili and Fattahi (2015)	Three-stage assembly flow-shop scheduling	Setup times		Single machine	Flow time, tardiness/earliness
Lei et al. (2022)	Distributed three-stage ASP			Single machine	Tardiness
Wang et al. (2023)	Distributed three-stage flow-shop scheduling	Setup times		Single machine	Makespan/tardiness
Fernandez-Viagas et al. (2022)	Multi-Stage assembly flow-shop scheduling			Single machine	Makespan
Sheikh et al. (2019)	Multi-Stage assembly flow-shop scheduling	Setup times, release time		Single machine	Makespan/total completion time
This paper	Assembly job-shop scheduling	Lot streaming, setup times	Batch transfer	Mixed-model assembly	Total completion time/inventory costs/setup costs/transport costs

a time. Comparatively speaking, the former is simpler and easier to implement. However, when a huge number of items need to be transferred and the unit transportation cost is expensive, adopting batch transfer is more realistic. A review of the job shop or flow-shop scheduling problems considering transfer can be found in the work by (Liu et al., 2019). Here, only the batch transfer in ASP is summarized. Regarding the batch transfer of finished products to customers, Basir et al. (2018), Kazemi et al. (2017) and Wang et al. (2016) respectively investigated the two-stage assembly flow-shop scheduling problem where the finished products need to be delivered to customers in batches. Yang and Xu (2021) addressed the distributed assembly permutation flow-shop scheduling problem, in which the finished products

from different factories were grouped into batches and delivered to customers. Hajibabaei and Behnamian (2023) studied flexible assembly job shop scheduling with machine breakdown and batch delivery. In these works, a new decision, namely the finished product allocations to batches, was considered, and delivery costs were usually selected as one of the performance criteria.

Regarding the batch transfer of intermediate products from the processing stage to the assembly stage (Hatami et al., 2010; Shoaardebili and Fattahi, 2015), respectively addressed the three-stage assembly flow-shop scheduling problem considering sequence-dependent setups and machine availability constraints (Lei et al., 2022). solved the distributed three-stage assembly scheduling problem, where each

factory contained one transport device (Jia et al., 2023). extend the work of (Lei et al., 2022) by considering flexible preventive maintenance (Wang et al., 2023). solved the distributed three-stage assembly flow shop scheduling with factory eligibility, transportation capacity, and setup time (Fernandez-Viagas et al., 2022). investigated a multi-stage assembly flow-shop scheduling problem that includes processing, collection, transport, assembly, and other stages. It can be seen that the existing works focused more on three-stage assembly flow-shop scheduling problems while few of them considered three-stage assembly job shop scheduling problems. Moreover, these works limited that only the components from a product were transferred together.

2.3. Mixed-model assembly

In the assembly stage, most studies assumed that all assembly operations are implemented on a single machine or parallel machines. For the case of the single machine, the assembly stage is regarded as a single scheduling problem with release times. In this regard, Komaki and Kayvanfar (2015) solved the assembly flow-shop scheduling with release time, and the “first completed first assembly” rule was adopted in the assembly stage. Fattahi et al. (2020) investigated a flexible assembly job shop scheduling problem to minimize makespan by simultaneously optimizing the processing machine selection and production sequence of processing and assembly. For distributed ASP, most studies considered that the assembly stage contained one or more assembly factories with each one having only one assembly machine. Wu et al. (2018) addressed the distributed assembly flexible job shop scheduling problem with the total production costs and earliness minimized. Yu et al. (2023) and Zhao et al. (2022) respectively investigated the distributed assembly permutation flow-shop and the distributed assembly blocking flow-shop scheduling problems with total flowtime criterion. Zhao et al. (2021) studied the distributed assembly no-wait flow-shop scheduling problem to minimize makespan.

For the case of parallel machines, two decisions are considered in the assembly stage, namely the assembly sequence and assembly machine assignment. In this regard, Wang et al. (2021) investigated the assembly job shop scheduling with uncertain assembly time, and a dual Q-learning is designed with assembly machine assignment optimized by the top-level, and sequence schedule determined by the bottom-level. Ren et al. (2021) addressed the energy-aware flexible assembly job shop scheduling problem. Al-Anzi and Allahverdi (2013) investigated the assembly flow-shop scheduling with two identical assembly machines to minimize total completion time. Tang and Zhang (2021) extended the above work by considering multiple identical assembly machines and preventive maintenance activities. Wang and Wang (2022) investigated the distributed assembly flow-shop considering multiple heterogeneous parallel assembly machines.

In fact, the above case of the single machine or parallel machines cannot completely reflect and adapt to actual production, in which mixed-model assembly lines are widely adopted to assemble similar heterogeneous products. At present, numerous works have been done on mixed-model assembly line balancing problems (Meng et al., 2022), mixed-model assembly line sequencing problems (Guo and Ryan, 2021), and so on. However, for the ASP with mixed-model assembly, only two studies have been conducted to the best of our knowledge. Among them, Zhang et al. (2021) addressed assembly permutation flow-shop scheduling with mixed-model assembly and maintenance to minimize makespan and maintenance costs. Cheng et al. (2022) solved the assembly job shop scheduling with mixed-model assembly to optimize total completion time and inventory time.

The above studies reveal that lot streaming, batch transfer, and mixed-model assembly are all very important for ASP. Nevertheless, it can be seen from Table 1 that no work has been done on simultaneously integrating all of them or even two of them to achieve effective and synchronous optimization of production and transfer.

3. Model formulation

The studied MAJSP-LS-BT with lot streaming in the processing stage, mixed-model assembly in the assembly stage, and batch transfer of components between the processing and assembly stages is described below. There are P types of products, M processing machines, a group of transport devices, and one mixed-model assembly line. Each type of product $p \in P$ has a single-level assembly structure and is assembled from $|C_p|$ components. The demand for product type p is l_p , and accordingly each type of component $c \in |C_p|$ has l_p identical components as a lot.

In the processing stage, all components are performed on M machines according to a specific processing route. Lot streaming is applied to accelerate the flow of production. Each lot is split into multiple smaller equal and consistent sublots. Sublots from different lots can be processed with intermingling. Sequence-dependent setup time s_{pcpc} exists between two successive sublots from different lots. After the completion of processing operations, components from different products are aggregated into a batch and transferred together to the assembly stage by any one of the idle transport devices. The total weight of components in a transfer batch cannot exceed the load capacity. In the assembly stage, components of each product are assigned to W stations on the mixed-model assembly line and assembled on these workstations according to mixed-model flow production. Some basic assumptions are listed below:

- (1) Equal and consistent sublots with intermingling is adopted.
- (2) Adequate transport devices are available.
- (3) The load capacity of transport devices is known.
- (4) Assembly station assignment of components is known.
- (5) Assembling multiple components of a product at one station is taken as one assembly operation.
- (6) At most one machining/assembly operation is performed at a time.
- (7) Each machining/assembly operation is executed exactly once, and preemption is not allowed.

3.1. Notations

Indexes:

p	The index of product type, $p = 1, 2, 3, \dots, P $.
c	Component index, $c = 1, 2, 3, \dots, C_p $.
o	The index of processing operations, $o = 1, 2, 3, \dots, O_{cp} $.
d	Sublot index, $d = 1, 2, 3, \dots, D $.
t	The index of transfer batches, $t = 1, 2, 3, \dots, T $.
n	The index of the component within a sublot, $n = 1, 2, 3, \dots, N $.
m	Machine index, $m = 1, 2, 3, \dots, M $.
w	Station index, $w = 1, 2, 3, \dots, W $.
k	The position index on each machine, $k = 1, 2, 3, \dots, K $.
h	The position index on each station, $h = 1, 2, 3, \dots, H $.
u_{pc}	Component c of product type p .
e_{pdn}	The n -th product of the d -th sublot of product type p .

Parameters:

δ_{mocp}	Processing time of operation o of u_{pc} on machine m .
l_p	Lot size of product type p .
s_{pcpc}	Setup time between u_{pc} and u_{pc} . If $p = p'$ and $c = c'$, $s_{pcpc} = 0$.
s_{pc}^0	Setup time for a machine to start processing u_{pc} .
ρ_{cp}	The weight of u_{pc} .
τ	Transport time of each transfer batch.
∂	The load capacity of transport devices.
b_{cpw}	If component c of product type p is assigned to station w , $b_{cpw} = 1$; else, $b_{cpw} = 0$.
β_{pw}	Assembly time of product type p on station w .
$\theta_1, \theta_2, \theta_3$	Cost coefficient of inventory, setup and transportation.
φ	A big positive number.

Binary variables:

X_{mkpdco}	X_{mkpdco} takes 1, if j_{pdco} is performed at the k -th position of machine m .
Z_{pd}	Z_{pd} takes 1, if product type p exists sublots d .

(continued on next page)

(continued)

Ψ_{pdn}	Ψ_{pdn} takes 1, if the d -th subplot of product type p exists the n -th product.
Q_t	Q_t takes 1, if the t -th transfer batch is activated.
I_{pdnct}	I_{pdnct} takes 1, if component c of e_{pdn} is assigned to the t -th transfer batch.
Y_{pdnh}	Y_{pdnh} takes 1, if e_{pdn} is assembled at the h -th position.
Positive variables:	
G_{pd}	Sublot size of the d -th subplot of product type p .
Continuous variables:	
Λ_{pdco}^s	The processing start time of the o -th operation of the d -th subplot of u_{pc} .
Λ_{pdnc}^e	The processing finish time of component c of e_{pdn} .
B_{pdnw}^e	The assembly finish time of e_{pdn} on station w .
V_{mk}^s	The start time of the k -th event position of machine m .
A_{wh}^s	The start time of the h -th event position of station w .
A_{wh}^e	The end time of the h -th event position of station w .
Π_{pdnc}	The inventory time of component c of e_{pdn} .

3.2. Objectives

Two types of objectives are considered. The first objective in Eq. (1) is to minimize the total completion time. The second objective in Eq. (2) is to minimize the total production costs including WIP inventory costs, setup costs, and transport costs. Among them, WIP inventory costs depend on the total inventory time of components calculated by Eqs. (3)–(5). Setup costs rely on the total setup time calculated by Eq. (6). Transport costs rely on the number of transfer batches defined in Eq. (7).

$$\min obj_1 \geq \sum_p \sum_d \sum_n \sum_c B_{pdnw}^e | \quad (1)$$

$$\min obj_2 \geq f_1 \bullet \theta_1 + f_2 \bullet \theta_2 + f_3 \bullet \theta_3 \quad (2)$$

$$f_1 = \sum_p \sum_d \sum_n \sum_c \Pi_{pdnc} \quad (3)$$

$$\Pi_{pdnc} \leq B_{pdnw}^e - \Lambda_{pdnc,1}^s - \beta_{pw} - \tau - \sum_m \sum_{o \in O_{cp}} \delta_{mopc} + \varphi \bullet (1 - \Psi_{pdn}), \forall p, c, w, b_{cpw} = 1 \quad (4)$$

$$\Pi_{pdnc} \geq B_{pdnw}^e - \Lambda_{pdnc,1}^s - \beta_{pw} - \tau - \sum_m \sum_{o \in O_{cp}} \delta_{mopc} - \varphi \bullet (1 - \Psi_{pdn}), \forall p, c, w, b_{cpw} = 1 \quad (5)$$

$$f_2 = \sum_m \sum_k \sum_p \sum_c \sum_d \sum_o \sum_p \sum_c \sum_d \sum_o X_{mkpdc} \bullet X_{m,k+1,p',c',d'} \bullet s_{pcp'c'} + \sum_m \sum_p \sum_c \sum_d \sum_o X_{m,1,pdc} \bullet s_{pc}^0 \quad (6)$$

$$f_3 = \sum_{t \in T} Q_t \quad (7)$$

3.3. Constraints

3.3.1. Processing constraints considering lot streaming and setups

Eqs. 8–13 are subplot split constraints. Specifically, Eq. (8) limits that the ahead subplot is first selected to accommodate components. Eqs. 9 and 10 denote that if a subplot exists, its subplot size is a positive value; otherwise, zero. Eq. (11) means that for each lot, its lot size is equal to the sum of subplot size. Eqs. 12 and 13 jointly achieve that the sublots from a lot have identical subplot size.

Eqs. 14–19 are subplot sequence constraints. Specifically, Eqs. 14 and 15 demands that each processing operation of sublots is performed once, and at most one processing operation exists in each event position. Eq. (16) means that processing operations are always first assigned to the ahead event position of machines. Eq. (17) denotes that for a given subplot, its next operation cannot start until its previous operation has been finished. Eqs. 18 and 19 associate the start time of sublots with that of the corresponding event positions.

Eqs. 20 and 21 are sequence-dependent setup constraints. Among them, Eq. (20) gives the setup constraint for those sublots processed at the first event position of machines. Eq. (21) gives the setup constraint

for two adjacent operations performed on a machine.

$$Z_{pd} \geq Z_{p,d+1}, \forall p, d < |D| \quad (8)$$

$$Z_{pd} \geq G_{pd} / \varphi, \forall p, d \quad (9)$$

$$Z_{pd} \leq G_{pd}, \forall p, d \quad (10)$$

$$\sum_d G_{pd} = l_p, \forall p \quad (11)$$

$$G_{pd} \leq G_{p,d+1} + \varphi \bullet (2 - Z_{pd} - Z_{p,d+1}), \forall p, d < |D| \quad (12)$$

$$G_{pd} \geq G_{p,d+1} - \varphi \bullet (2 - Z_{pd} - Z_{p,d+1}), \forall p, d > 1 \text{ and } d < |D| \quad (13)$$

$$\sum_m \sum_k X_{mkpdc} = Z_{pd}, \forall p, d, c, o \quad (14)$$

$$\sum_p \sum_d \sum_c \sum_o X_{mkpdc} \leq 1, \forall m, k \quad (15)$$

$$\sum_p \sum_d \sum_c \sum_o X_{mkpdc} \geq \sum_p \sum_d \sum_c \sum_o X_{m,k+1,pdc}, \forall m, k < |K| \quad (16)$$

$$\Lambda_{pdco}^s + \sum_m \delta_{mopc} \bullet G_{pd} \leq \Lambda_{pdco+1}^s + \varphi \bullet (1 - Z_{pd}), \forall p, d, c, o < |O_{cp}| \quad (17)$$

$$\Lambda_{pdco}^s \leq V_{mk}^s + \varphi \bullet (1 - X_{mkpdc}), \forall m, k, p, d, c, o \quad (18)$$

$$\Lambda_{pdco}^s \geq V_{mk}^s - \varphi \bullet (1 - X_{mkpdc}), \forall m, k, p, d, c, o \quad (19)$$

$$V_{mk}^s \geq \sum_p \sum_d \sum_c \sum_o X_{mkpdc} \bullet s_{pc}^0, \forall m, k = 1 \quad (20)$$

$$V_{mk}^s + \sum_p \sum_d \sum_c \sum_o \sum_p \sum_d \sum_c \sum_o s_{pcp'c'} \bullet X_{mkpdc} \bullet X_{m,k+1,p',d',c'} + \sum_p \sum_d \sum_c \sum_o \delta_{mopc} \bullet G_{pd} \bullet X_{mkpdc} \leq V_{m,k+1}^s, \forall m, k < |K| \quad (21)$$

3.3.2. Batch transfer constraints

Eq. (22) means that each component is assigned to only one transfer batch. Eq. (23) guarantees that the ahead transfer batch is activated first. Eq. (24) ensures that only those activated transfer batches can transfer components. Eq. (25) limits the total weight of components assigned to a transfer batch.

$$\sum_{t \in T} I_{pdnct} = \Psi_{pdn}, \forall p, c, d, n \quad (22)$$

$$Q_t \geq Q_{t+1}, \forall t < |T| \quad (23)$$

$$\sum_{p \in P} \sum_{c \in C_p} \sum_{d \in D} \sum_{n \in N} I_{pdnct} \leq \varphi \bullet Q_t, \forall t \quad (24)$$

$$\sum_{p \in P} \sum_{c \in C_p} \sum_{d \in D} \sum_{n \in N} I_{pdnct} \bullet \rho_{cp} \leq \vartheta, \forall t \quad (25)$$

3.3.3. Mixed-model assembly constraints

Similar to Eqs. 14 and 15, Eqs. 26 and 27 constrain the execution of each assembly operation. Eq. (28) means that for each event position, its end time is equal to its start time plus the assembly time. Eq. (29) gives the start and end time relation of two event positions on adjacent stations. Eq. (30) limits that the next event position cannot start until its previous one has been finished. Eqs. 31 and 32 associate the assembly finish time of products with the end time of the corresponding event positions.

$$\sum_h Y_{pdnh} = \Psi_{pdn}, \forall p, d, n \quad (26)$$

$$\sum_p \sum_d \sum_n Y_{pdnh} \leq 1, \forall h \quad (27)$$

$$A_{wh}^s + \sum_p \sum_d \sum_n \beta_{pw} \bullet Y_{pdnh} \leq A_{wh}^e, \forall w, h \quad (28)$$

$$A_{wh}^e \leq A_{w+1,h}^s, \forall w < |W|, h \quad (29)$$

$$A_{wh}^e \leq A_{w,h+1}^s, \forall w, h < |H| \quad (30)$$

$$B_{pdnw}^e \leq A_{wh}^e + \varphi \bullet (1 - Y_{pdnh}), \forall p, d, n, w, h \quad (31)$$

$$B_{pdnw}^e \geq A_{wh}^e - \varphi \bullet (1 - Y_{pdnh}), \forall p, d, n, w, h \quad (32)$$

3.3.4. Spatial-temporal links among three stages with differentiated production lot size

The studied problem contains three stages with differentiated production lot size. In the processing stage, identical components are grouped into a small subplot and processed together. In the transfer stage, multiple components are assigned to a batch and transferred together. In the assembly stage, the single-piece flow is applied to assemble products one by one. To construct spatial-temporal links among the three stages, the finish time of each component within a subplot is first derived by Eq.33 and 34. Here, Ψ_{pdn} denotes the index of components in a subplot and Ψ_{pdn} obeys constraints (35–36). Eq. (35) limits the order of components in a subplot. Eq. (36) ensures that the number of components in a subplot is equal to subplot size.

Eq. (37) means that the start time of a transfer batch must be later than the finish time of any one component assigned to this transfer batch. Eq. (38) demands that each station can start only when all components assigned to this station have arrived.

$$\Lambda_{pdnc}^e \leq \Lambda_{pdco}^s + n \bullet \sum_m \delta_{mopc} + \varphi \bullet (1 - \Psi_{pdn}), \forall p, d, c, n, o = |O_{cp}| \quad (33)$$

$$\Lambda_{pdnc}^e \geq \Lambda_{pdco}^s + n \bullet \sum_m \delta_{mopc} - \varphi \bullet (1 - \Psi_{pdn}), \forall p, d, c, n, o = |O_{cp}| \quad (34)$$

$$\Psi_{pdn} \geq \Psi_{pd,n+1}, \forall p, d, n < |N| \quad (35)$$

$$\sum_n \Psi_{pdn} = G_{pd}, \forall p, d \quad (36)$$

$$\Lambda_{pdnc}^e - \varphi \bullet (1 - I_{pdnct}) \leq \Gamma_t^s, \forall p, c, o = |O_{cp}|, t \quad (37)$$

$$\Gamma_t^s + \tau - \varphi \bullet (2 - Z_{cpt} - Y_{pdnh}) \leq A_{wh}^s, \forall p, c, t, w \quad (38)$$

3.4. Model linearization

The above model contains two types of nonlinear terms: $G_{pd} \bullet X_{mkpdc}$ and $X_{mkpdc} \bullet X_{m,k+1,p'd'c'o}$. To eliminate non-linearity, two sets of linearization methods are presented below.

To replace $G_{pd} \bullet X_{mkpdc}$, an intermediate variable I_{mkpdc}^{XG} is introduced, and Eqs.39–42 are added to ensure that when X_{mkpdc} takes 0, I_{mkpdc}^{XG} takes 0; when X_{mkpdc} takes 1, I_{mkpdc}^{XG} takes G_{pd} .

$$I_{mkpdc}^{XG} \leq \varphi \bullet X_{mkpdc}, \forall m, k, p, d, c, o \quad (39)$$

$$I_{mkpdc}^{XG} \geq -\varphi \bullet X_{mkpdc}, \forall m, k, p, d, c, o \quad (40)$$

$$I_{mkpdc}^{XG} \leq G_{pd} + \varphi \bullet (1 - X_{mkpdc}), \forall m, k, p, d, c, o \quad (41)$$

$$I_{mkpdc}^{XG} \geq G_{pd} - \varphi \bullet (1 - X_{mkpdc}), \forall m, k, p, d, c, o \quad (42)$$

$X_{mkpdc} \bullet X_{m,k+1,p'd'c'o}$ is replaced by a binary variable $I_{mkpdcop'd'c'o}^{XX}$, and $I_{mkpdcop'd'c'o}^{XX}$ obeys constraints (43–45).

$$I_{mkpdcop'd'c'o}^{XX} \leq X_{mkpdc}, \forall m, k, p, d, c, o, p', d', c', o' \quad (43)$$

$$I_{mkpdcop'd'c'o}^{XX} \leq X_{m,k+1,p'd'c'o}, \forall m, k, p, d, c, o, p', d', c', o' \quad (44)$$

$$I_{mkpdcop'd'c'o}^{XX} \geq X_{mkpdc} + X_{m,k+1,p'd'c'o} - 1, \forall m, k, p, d, c, o, p', d', c', o' \quad (45)$$

4. Improved coevolutionary simulated annealing algorithm

The studied problem includes three stages. Each stage involves at least one decision, and each decision has a different influence on the optimization objective. For this multi-decision problem, the coevolutionary algorithm which can effectively achieve the independent evolution and global optimization of multiple sub-problems is adopted. The coevolutionary algorithm was first introduced by Potter (Potter and de Jong, 2000), and it utilized the idea of divide-and-conquer to divide a complex problem into multiple sub-problems. Each sub-problem evolves separately, but each individual in each sub-problem is evaluated by interacting with the individuals from other sub-problems to achieve coevolution. In this work, according to the production stage, the studied problem is divided into three sub-problems. Each sub-problem evolves via the simulated annealing algorithm (SA) (Cheng et al., 2022). Overall solutions are created by concatenating the individuals from three sub-problems. For each sub-problem, its individuals are evaluated by interacting with the other two individuals in the best overall solution.

In addition, to improve the performance, two-stage decoding, rule-based initialization, adaptive execution strategy of sub-problems, and restart mechanism are designed. The improved coevolution simulated annealing algorithm is denoted as ICSEA for short.

4.1. Solution encoding

For the studied problem, each solution consists of five parts: lot split, subplot sequence, number of transfer batches, transfer batch assignment, and assembly sequence. Thus, a five-layer encoding is presented below.

The lot split segment determines the subplot quantity for each type of product. For the instance with P types of products, this segment is represented by P elements and each element takes a value from the interval $[1, l_p]$, where l_p denotes the lot size of product type p .

The subplot sequence segment determines the processing order of subplots. To adapt to any lot split scheme, the subplot sequence corresponding to the maximum number of subplots is considered. By treating each subplot as an individual or a job, the maximum subplot sequence can be created according to the widely used processing sequence encoding for job shop scheduling problems (Cheng et al., 2022). For a given subplot split scheme, we first determine the required subplots (called actual subplots) according to the subplot quantity, and then sort out them according to the maximum subplot sequence. As shown in Fig. 1, the maximum subplot sequence segment determines the processing order of $\sum_{p=1}^{|P|} l_p \bullet |C_p| \bullet |O_{cp}|$ subplots. If the subplot quantity of product p is q_p , the actual subplots contain $\sum_{p=1}^{|P|} q_p \bullet |C_p| \bullet |O_{cp}|$ elements, and the indexes of these elements in the maximum subplot sequence segment represent their processing order.

The number of transfer batches $|T|$ is encoded with one gene. Since each transfer batch has at least one component and the load capacity is δ , $|T|$ should be less than the number of components $\sum_{p=1}^{|P|} l_p \bullet |C_p|$ but greater than the ratio of total component weight to load capacity $(\sum_{p=1}^{|P|} \sum_{c=1}^{|C_p|} l_p \bullet \rho_{cp}) / \delta$.

The transfer batch assignment determines the assigned transfer batch of each component. This segment contains $\sum_{p=1}^{|P|} l_p \bullet |C_p|$ genes with each gene represented by a transfer batch index. Each transfer batch index takes a value from the interval $[1, |T|]$.

The assembly sequence segment determines the assembly order of products. This segment is encoded with a permutation $\Phi_n = \{\varphi_1, \dots, \varphi_j, \dots, \varphi_n\}$, where φ_j represents the product index and n denotes the number of products, and $n = \sum_{p=1}^{|P|} l_p$. The product that appears ahead in the permutation is assembled first.

To demonstrate the above encoding, a small case is provided. This case contains two types of products with each type having four copies.

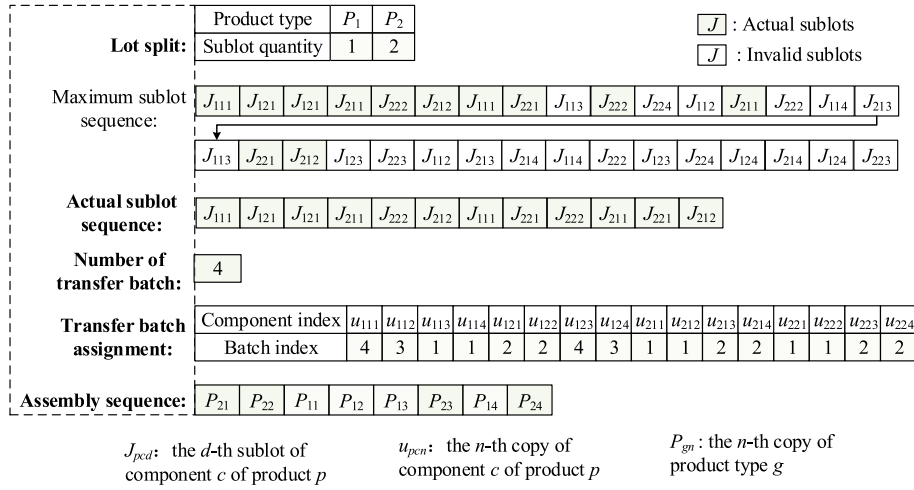


Fig. 1. Solution representation.

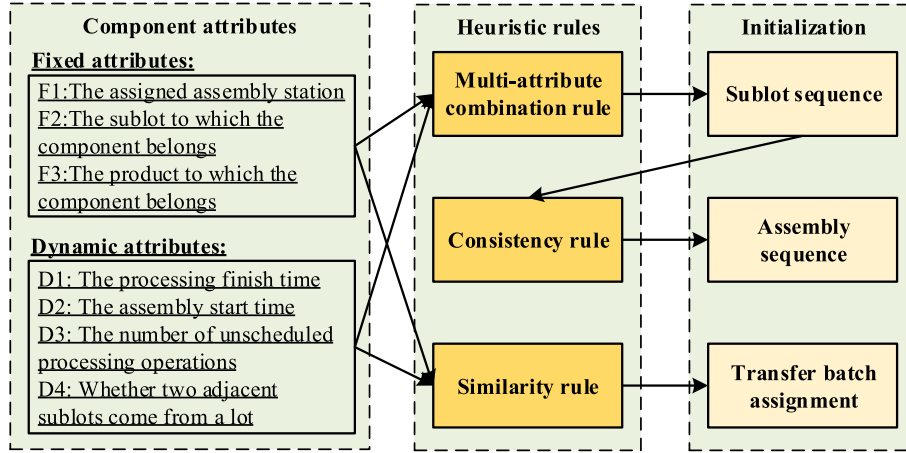


Fig. 2. Rule-based initialization.

Each product consists of two components having two processing operations. Thus, the maximum subplot is $2 \times 4 \times 2 = 16$, and the maximum number of processing operations is $16 \times 2 = 32$. A feasible solution for this case is encoded as shown in Fig. 1. It can be seen that the two types of products are respectively split into one and two sublots. For such a lot split scheme, the actual processing operations is $(1 \times 2 + 2 \times 2) \times 2 = 12$, and their processing sequence is $\{J_{111}, J_{121}, J_{121}, J_{211}, J_{222}, J_{212}, J_{111}, J_{221}, J_{222}, J_{211}, J_{221}, J_{212}\}$. In addition, all processed components are transferred in four batches. Taking the 4th batch as an example, two components $\{u_{111}, u_{123}\}$ are transferred by the 4th batch. In the assembly stage, P_{21} is assembled first and followed by P_{22} .

4.2. Two-stage decoding

The decoding process aims to calculate the total completion time and total production costs including the transport costs, setup costs, and inventory costs. Among them, the transport costs can be obtained based on the number of transfer batches in the encoding. The setup costs depend on the sequence-dependent setup time and can be derived after the subplot sequence is given.

To obtain the total completion time and inventory costs, a feasible schedule is generated via the classical semi-active decoding. As shown in **Algorithm 1**, the essence of semi-active decoding is to schedule all tasks including the processing operations, transfer batches, and assembly operations as early as possible under all the required constraints satisfied. Especially, as demonstrated in (Alfieri et al., 2023), the schedule created by the semi-active decoding is optimal for the minimization of regular measures such as total completion time.

However, as an irregular measure, the obtained inventory costs can be further reduced by shifting some operations to the right. Thus, a right-shift decoding is designed as shown in **Algorithm 2** to adjust the above schedule without extending the obtained total completion time. In brief, take the obtained completion time of each product as the base point, from the assembly stage to the transport stage and then to the processing stage, and shift the relevant tasks to the right as far as possible to reduce the flow time of each component, so as to reduce inventory costs.

Algorithm 1. Semi-active decoding

Input: A solution.

Output: The total completion time and total production costs of the solution.

01: Set the available time of machines $T_{1 \times |M|}^M \leftarrow \text{zeros}(1, |M|)$, workstations $T_{1 \times |W|}^W \leftarrow \text{zeros}(1, |W|)$, transfer batches $\Gamma_{1 \times t}^T \leftarrow \text{zeros}(1, |T|)$, components $T_{1 \times F}^C \leftarrow \text{zeros}(1, F)$, sublots $T_{1 \times R}^S \leftarrow \text{zeros}(1, R)$, and products $T_{|P| \times |W|}^P \leftarrow \text{zeros}(|P|, |W|)$. Here, $F = \sum_{p=1}^{|P|} l_p \cdot |C_p| \cdot |O_{cp}|$, $R = \sum_{p=1}^{|P|} q_p \cdot |C_p|$, $U = \sum_{p=1}^{|P|} q_p \cdot |C_p| \cdot |O_{cp}|$, and other notations are listed in section 3.1

02: **For** $j=1: U$ // determine the earliest finish time of each subplot.

03: take the j -th subplot r from the actual subplot sequence segment.

04: find the processing machine m of subplot r and calculate the setup time st .

05: $T_{1,r}^S \leftarrow \max([st + T_{1,m}^M, T_{1,r}^S])$.

06: $T_{1,m}^M \leftarrow T_{1,r}^S + \text{processing time of subplot } r$.

07: **For** $h=1: H$ // determine the earliest finish time of each component within a subplot.

08: $T_{1,h}^C \leftarrow T_{1,r}^S + \text{component index} \cdot \text{processing time of component } h$.

09: **Endfor**

10: $T_{1,r}^S \leftarrow T_{1,m}^M$

11: **Endfor**

12: **For** $t=1: |T|$ // determine the earliest finish time of each transfer batch.

13: find all the components Ω assigned to the t -th batch.

14: $\Gamma_{1,t}^T \leftarrow \max_{c^* \in \Omega} T_{1,c^*}^C + \text{transportation time}$.

15: **Endfor**

16: **For** $p=1: |P|$ //determine the earliest finish time of each product on each assembly workstation.

17: take the p -th product p^* from the product sequence segment.

18: **For** $w=1: |W|$

19: find the components c^* of product p^* assembled on workstation w .

20: find the transfer batch t^* of components c^* .

21: $T_{1,w}^W \leftarrow \max(T_{p^*,w}^P, T_{1,w}^W, \Gamma_{1,t^*}^T) + \text{assembly time}$; $T_{p^*,w}^P \leftarrow T_{1,w}^W$.

22: **End for**

23: **Endfor**

24: calculate the total completion time, transport costs, setup costs, and inventory costs.

Algorithm 2. Right-shift decoding

Input: the earliest finish time of products $T_{p,|w|}^P$, transfer batches Γ_t^T and components $T_{1 \times F}^C$.

Output: the modified inventory costs and the adjusted finish time of $T_{p,|w|}^P$, transfer batches $\Gamma_{1 \times t}^T$ and components $T_{1 \times F}^C$.

01: Let the available time of workstations $T_{1 \times |W|}^W \leftarrow \text{ones}(1, |W|) \cdot \max(T_{p,|w|}^P)$, transfer batches $\Gamma_{1 \times t}^T \leftarrow \text{ones}(1, |T|) \cdot \max(T_{p,|w|}^P)$, machines $T_{1 \times |M|}^M \leftarrow \text{ones}(1, |M|) \cdot \max(T_{p,|w|}^P)$, products $T_{1 \times p}^P \leftarrow T_{p,|w|}^P$ and components $T_{1 \times F}^C \leftarrow \text{ones}(1, F) \cdot \max(T_{p,|w|}^P)$.

02: **For** $p = |P| : 1$

03: From $T_{p,|w|}^P$, find the product p with the largest finish time, and set $T_{p,|w|}^P \leftarrow 0$.

04: **For** $w = |W| : 1$

05: $T_{1,w}^W \leftarrow \min(T_{1,p}^P, T_{1,w}^W) - \text{assembly time of } p$; $T_{p,w}^P \leftarrow T_{1,w}^W$.

06: **End for**

07: **End for**

08: **For** $t=1: |T|$

09: Find all the products Ω whose components are transported by the t -th batch.

10: $\Gamma_{1,t}^T \leftarrow \min_{p^* \in \Omega} T_{p^*,w}^P - \text{transportation time}$.

11: **Endfor**

12: **For** $j = 1 : U$

13: from $T_{1 \times F}^C$, find the operation k with the largest finish time.

14: find the processing machine m , the assigned transfer batch t , the component d , and product i of k .

15: calculate the setup time st .

16: $T_{1,k}^C \leftarrow \min(T_{1,m}^M, T_{1,k}^C, \Gamma_{1,t}^T) - \text{processing time of } k - st$.

17: $T_{1,m}^M \leftarrow T_{1,k}^C$, $T_{1,k}^C = 0$.

18: **End for**

19: recalculate inventory costs.

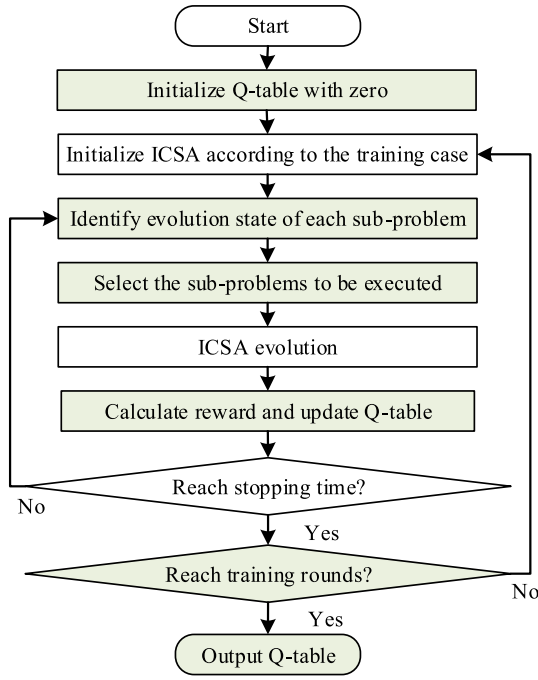


Fig. 3. Training procedure of Q-learning for the adaptive execution of sub-problems.

In fact, executing the right-shift decoding can significantly inventory costs, but it is time-consuming. Thus, in each run of algorithms, the right-shift decoding is performed only on the final Pareto frontier solutions.

4.3. Rule-based initialization

As shown in Fig. 2, by mining component attributes, three heuristic rules are developed and applied to achieve the initialization of subplot sequence, assembly sequence, and transfer batch assignment.

For the subplot sequence, a multi-attribute combination rule is developed. To determine the attributes that most affect the performance of subplot sequence, a large number of problem-related attributes such as processing time, and remaining processing time are tested on a set of instances and evaluated by Eq. (46). Here, n denotes the number of testing instances. $obj_{k,j}$ represents the k -th objective value obtained when applying the rule constructed by attribute j to instance i . $\max(obj_{1,i} + obj_{2,i})$ and $\min(obj_{1,i} + obj_{2,i})$ respectively represent the worst and the best results obtained by all the attributes for instance i .

$$f_j = \sum_{i=1}^n \frac{(obj_{1,i,j} + obj_{2,i,j}) - \min(obj_{1,i} + obj_{2,i})}{\max(obj_{1,i} + obj_{2,i}) - \min(obj_{1,i} + obj_{2,i})} \quad (46)$$

Through Eq. (46), we found that the number of unscheduled processing operations α performs the best, followed by the assigned assembly station β and the adjacent subplot γ . Given this, a multi-attribute combination rule is defined as follows. The subplot with the largest α is processed first. If there are multiple sublots with the same α , the subplot with a smaller β is processed first. When these sublots are assembled on the same station, the subplot which comes from the same lot as its adjacent subplot is processed preferentially. When all of the above conditions are the same, a subplot is randomly selected.

For the assembly sequence, a consistency rule is designed. By endowing the product whose components are finished first with a higher assembly priority, the processing sequence and assembly sequence can maintain a high degree of consistency. As a result, those finished components can be assembled as soon as possible. In this work, the assembly priority is obtained through the following two steps: (1) according to the created subplot sequence, the semi-active schedule is applied to obtain

the processing finish time of each component; (2) calculate the average processing finish time of components of each product, and sort them to create the assembly priority.

For the transfer batch assignment, a similarity rule is developed to assign those components with similar attributes to a transfer batch. Attributes related to batch transfer include the subplot to which the component belongs, the assigned assembly workstation, the product to which the component belongs, the processing finish time, and the assembly start time. The first three attributes are fixed parameters, while the last two attributes are dynamic and determined through the semi-active schedule. Here, the semi-active schedule is implemented under the assumption that each component is assigned to a transfer batch. After the values of the above five attributes of each component are obtained, k -means clustering, an unsupervised machine learning technique that is effective in grouping objects into clusters based on similarity, is applied to form component clusters. Each transfer batch is responsible for a component cluster. Details about the k -means clustering can refer to (Sinaga and Yang, 2020).

As for the subplot quantity and the number of transfer batches, they are created by randomly taking a feasible value from the given interval.

4.4. Adaptive execution strategy of sub-problems

In the classical coevolution algorithms, the evolution state of sub-problems is not concerned, and all sub-problems are performed in turn in each iteration. In fact, the size of the solution space of different sub-problems may be significantly different. In this case, some sub-problems with small solution space may already converge or fall into local optima, while other sub-problems still need to invest a lot of computing resources to explore more potential solutions. Given this, Q-learning is applied to adaptively select the sub-problems to be executed according to the evolution state of each sub-problem, so as to maximize the exploration efficiency.

As a widely used value-based reinforcement learning method, Q-learning self-learns the optimal action selection strategy by iteratively interacting with the environment. The interaction process is described below. An agent observes the current state s_t , and moves to the next state s_{t+1} by taking an action a_t . Meanwhile, a reward r is returned for the selected action, and a Q-table which stores the Q-value $Q_t(s_t, a_t)$ of each state-action pair is updated by Eq. (47). By repeating the above interaction, a final Q-table is created and the action with the largest Q-value is regarded as the best action.

$$Q_{t+1}(s_t, a_t) \leftarrow Q_t(s_t, a_t) + \lambda \cdot \left[r_t + \gamma \cdot \max_a Q_t(s_{t+1}, a) - Q_t(s_t, a_t) \right] \quad (47)$$

where s_t and s_{t+1} respectively denote the current state and the next state. a_t represents the action taken in the state s_t . λ and γ are the learning rate and discount factor.

To find an optimal adaptive execution strategy of sub-problems, three key components of Q-learning including the state set, action set, and reward function are defined below, and the training procedure of Q-table is presented in Fig. 3.

Action set: The action is to select the sub-problems to be executed at each iteration. The studied problem contains three sub-problems. If a sub-problem needs to be executed, it is marked as 1; Otherwise, 0. Thus, the action set consists of seven actions: $A1 = \{0, 0, 1\}$; $A2 = \{0, 1, 1\}$; $A3 = \{0, 1, 0\}$; $A4 = \{1, 0, 1\}$; $A5 = \{1, 0, 0\}$; $A6 = \{1, 1, 1\}$; $A7 = \{1, 1, 0\}$. Here, A1 means only sub-problem three needs to be executed.

State set: The evolution state is described from two aspects: the update status of the archive set S and the cumulative running time C . Among them, S takes 1 or 0. 1 denotes that new non-dominated solutions are obtained, and the archive set is updated. 0 represents that the archive set remains unchanged. C is discretized into three intervals: $\{[0, ST/3], [ST/3, ST/1.5], [ST/1.5, ST]\}$, where ST represents the given stopping time of algorithms. The three intervals can be viewed as the

early, middle, and late running of algorithms. The combination of C and S makes up six evolution states.

Reward function: As shown in Eqs. 48 and 49, the performance of the Pareto frontier solutions is first evaluated by the inverted generational distance (IGD) metric and then used to construct a reward function.

$$IGD = \frac{\sum_{x^* \in P^*} d(x^*, x)}{|P^*|} \quad (48)$$

$$r = IGD / (t_c - t_p) \quad (49)$$

where t_p and t_c respectively represent the cumulative running time before and after an action is executed. P^* denotes the Pareto frontier solutions obtained after the action is executed. $d(x^*, x)$ is the minimum Euclidean distance from the solution x^* to solution x (x denotes the Pareto frontier solution before executing the action).

4.5. Neighborhood operators

Swap, insertion, mutation, and flip are four widely used neighborhood operators. In this work, all of them are adopted to create neighbor solutions. Details about these operators can refer to (Chang et al., 2023; Zhang et al., 2021).

For the lot split, the mutation operator is applied to change subplot quantity. For the subplot sequence, four operators including actual subplot-oriented swap and insertion, and multi-swap and multi-insertion are designed. Here, the actual subplot-oriented swap and insertion mean that the subplot selected for swap or insertion must be from actual sublots since changing the position of invalid sublots cannot change fitness. The multi-swap and multi-insertion mean executing the swap and insertion operators I times, I represents the number of product types.

As for the number of transfer batches, the mutation operator is applied to increase or decrease one batch at a time. Three operators including batch swap, batch insertion, and batch mutation are used to change transfer batch assignment. Note that, executing the above operators may create infeasible transfer batch assignments, in which some batches may not contain components or exceed the load capacity. In this case, a repair mechanism is designed to randomly select one component at a time from the batch with the most components, and assign it to the batch with the least components.

For the assembly sequence, product swap, product insertion, and product flip are applied.

In each iteration, the above twelve neighborhood operators are randomly selected to create NS neighbor solutions, and the solution with the best fitness is recorded.

4.6. Archive update and acceptance criterion and restart mechanism

Archive set stores non-dominated solutions obtained during the evolution. The initial archive set is empty. In each iteration, the newly created solution is first added to the archive set, and then the dominant solution in the archive set is removed.

The original metropolis acceptance criterion in the SA is set for the single-objective optimization problem. Inspired by (Li et al., 2016), the metropolis acceptance criterion is adjusted as shown in Eq. (50) for the studied bi-objective problem. In this setting, the new solution can be directly accepted only if one of the two objectives in the new solution is superior to the corresponding objective of the incumbent solution, and the other objective is better or only slightly worse than that of the incumbent solution.

$$P = \begin{cases} 1, & \text{if } \left(\sum_{i=1}^2 obj_{n,i} - \sum_{i=1}^2 obj_{o,i} \right) < 0 \\ \exp \left(\frac{\sum_{i=1}^2 obj_{o,i} - \sum_{i=1}^2 obj_{n,i}}{T_c} \right), & \text{otherwise} \end{cases} \quad (50)$$

where T_c denotes the current temperature. $obj_{n,i}$ and $obj_{o,i}$ are respectively the i -th objective value of the newly created solution and the incumbent solution.

In addition, to avoid falling into local optima during the evolution of each sub-problem, a restart mechanism is introduced for each sub-problem. When a sub-problem has not obtained a non-dominated solution for RL successive iterations, a solution is randomly selected from the archive set to replace the current solution of this sub-problem, and the restart counter of this sub-problem is set to 0.

4.7. Framework of the proposed algorithm

Fig. 4 shows the framework of the proposed algorithm. The detailed procedures are described below.

- Step 1: Create one overall solution via the rule-based initialization.
- Step 2: Determine the sub-problems to be executed. Identify the evolution state, and find out the most appropriate sub-problem execution action from the trained Q-table.
- Step 3: If sub-problem one does not require evolution, go to step 4. Otherwise, perform the evolution of sub-problem one.
- Step 4: If sub-problem two does not require evolution, go to step 5. Otherwise, perform the evolution of sub-problem two.
- Step 5: If sub-problem three does not require evolution, go to step 6. Otherwise, perform the evolution of sub-problem three.
- Step 6: Randomly select one solution from the archive set to update the overall solution.
- Step 7: If the number of iterations is not reached, go to step 2. Otherwise, execute the cooling operation.
- Step 8: If the stopping time is not reached, go to step 2. Otherwise, output the Pareto frontier solutions. For each Pareto frontier solution, apply right-shift decoding to obtain a final schedule.

5. Experiment study

In this section, a series of comparison experiments of algorithms are first conducted to verify the effectiveness of the proposed algorithm. Then, four production scenarios are compared and analyzed to reveal the impact of lot streaming and batch transfer on mixed-model assembly job shop scheduling.

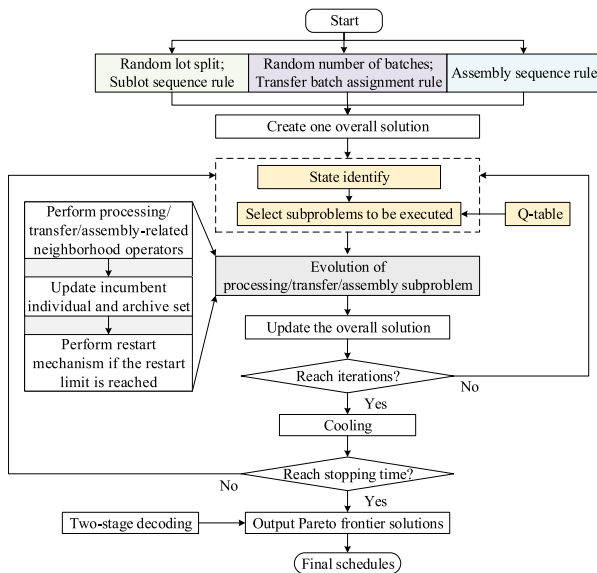


Fig. 4. Framework of the proposed IC-SA.

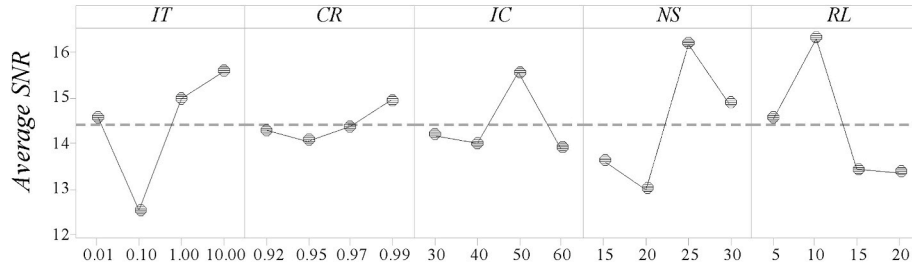


Fig. 5. The main effect plot of CSA-I.

Table 2

The best parameters of ICSA and its variants.

Parameters	Level	CSA	CSA-R	CSA-I	CSA-IR	ICSA
IT	{0.01, 0.1, 1, 10}	0.1	0.1	10	1	10
CR	{0.92, 0.95, 0.97, 0.99}	0.95	0.92	0.99	0.92	0.99
IC	{30, 40, 50, 60}	30	40	50	40	50
NS	{15, 20, 25, 30}	20	20	25	25	25
RL	{5, 10, 15, 20}	10	15	10	10	10

Table 3

The best parameters of other comparison algorithms.

Algorithms	Parameters	Level	value
MPMA-QL	Population size	{40, 60, 80, 100}	80
	Learning rate	{0.1, 0.2, 0.3, 0.4}	0.1
	Greedy factor	{0.1, 0.2, 0.3, 0.4}	0.2
	Discount factor	{0.7, 0.8, 0.9, 1}	0.8
TLBO	Population size	{40, 80, 120, 160}	120
	Number of classes	{4, 6, 8, 10}	6
	Teaching probability u_1	{0.1, 0.2, 0.3, 0.4}	0.4
	Teaching probability u_2	{0.6, 0.7, 0.8, 0.9}	0.9
MOSA	IT	{0.01, 0.1, 1, 10}	10
	CR	{0.92, 0.95, 0.97, 0.99}	0.95
	IC	{30, 40, 50, 60}	50
	NS	{15, 20, 25, 30}	20
NSGA-II	Population size	{40, 80, 120, 160}	80
	Crossover rate	{0.92, 0.95, 0.97, 0.99}	0.99
MOICA	Mutation rate	{0.1, 0.15, 0.17, 0.2}	0.2
	Population size	{50, 100, 150, 200}	100
MOTS	Number of empires	{5, 10, 15, 20}	10
	Possibility of revolution	{0.3, 0.4, 0.5, 0.6}	0.6
MOCABC	Tabu tenor	{3, 5, 8, 10}	8
	Neighborhood size	{5, 9, 15, 20}	15
MOCABC	Population size	{20, 40, 60, 80}	60
	Number of trials	{10, 20, 30, 40}	20

5.1. Experiment setting

In the work by (Cheng et al., 2022), a MAJSP dataset is created. This dataset contains six instances A1-A6 for parameter calibration and thirty-two instances B1-B32 for comparative experiments. Based on the MAJSP dataset, some lot-related and transport-related parameters are created to form a dataset for the studied MAJSP-LS-BT. Let U denote the uniform distribution. The lot size of each product type is set to obey $U[4, 8]$. The setup time between sublots obeys $U[10, 30]$. The weight of components obeys $U[5, 8]$. The load capacity and the transport time respectively take values of 40 and 10. Besides, the coefficients of setup cost, transport cost, and inventory cost respectively obey $U[30, 50]$, $U[60, 100]$, and $U[30, 50]$.

All algorithms are programmed with MATLAB and tested on a processor with Intel (R) Core (TM) i7-10700 CPU. For each run of algorithms, the stopping time is identical and set as 1×10^5 s. Here, I and N respectively denote the number of product types and the number of components of a product.

Table 4

Q-table for sub-problem execution under each state.

States	Actions						
	A1	A2	A3	A4	A5	A6	A7
(C1, S1)	0.24	0.34	0.18	0.11	0.23	0.14	1.94
(C1, S2)	0.57	0.77	0.56	2.89	1.09	0.63	0.92
(C2, S1)	0.06	0.09	0.10	0.11	0.39	0.08	0.10
(C2, S2)	0.18	0.10	0.12	0.06	0.11	0.43	0.06
(C3, S1)	0.18	0.17	0.19	0.20	0.30	0.14	0.19
(C3, S2)	0.04	0.02	0.01	0.02	0.05	0.01	0.14

To evaluate the performance of multi-objective algorithms, three popular multi-objective performance metrics: *IGD*, Hypervolume Ratio (*HVR*), and *C metric* are adopted. Definitions of the three metrics can be found in (Meng et al., 2022; Zhang et al., 2021). Regarding the *IGD* and *HVR*, the smaller the *IGD* and the larger the *HVR*, the better the convergence and diversity of algorithms. As for the *C metric*, the value of $C(A, B)$ closer to 1 means that more solutions by algorithm B are dominated by the solutions obtained by algorithm A.

5.2. Parameter calibration and Q-table training

For all the involved comparison algorithms, Taguchi experiments are conducted to identify their best parameters. Since the proposed ICSA needs to further train Q-learning on its variant CSA-I to obtain an adaptive execution strategy of sub-problems, the parameters of CSA-I are first calibrated, and then the obtained best parameters are directly transplanted to ICSA.

CSA-I contains five key parameters: initial temperature (*IT*), cooling rate (*CR*), iteration number under each temperature (*IC*), neighborhood size (*NS*), and restart limit (*RL*). Each parameter is tested at four levels, and a Taguchi orthogonal $L_{16}(4^5)$ is selected. For each one of the sixteen parameter combinations, the CSA-I runs on six calibration cases and outputs the Pareto frontier solutions after the stopping time 1×10^5 is reached. With the obtained Pareto frontier solutions, the *HVR* metric is calculated, and the average *HVR* on all calibration cases is used as the response value. Based on the obtained response values, the main effect plot for signal-to-noise ratio (*SNR*) is drawn in Fig. 5. For the *SNR*, the parameter level with the largest *SNR* is the best. Thus, the best parameters of CSA-I are $\{IT = 10; CR = 0.99; IC = 50; NS = 25; RL = 10\}$.

For other comparison algorithms, similar parameter calibration procedures are performed and their best parameters are presented in Tables 2 and 3.

After the best parameters of ICSA are determined, a Q-table is trained on case A6 for 40 rounds according to Fig. 3. Table 4 shows the training results. In Table 4, the actions recommended by Q-learning are shown in bold. It can be seen that all the recommended actions are concentrated in $\{A4, A5, A6, A7\}$. For these recommended actions, one striking feature is that the processing sub-problem is always executed. Two possible reasons for the recommended results are summarized below. (1) The processing sub-problem has a larger solution space than other sub-problems, and hence more computing resources should be allocated to

Table 5
Comparison results of the variants of ICSA.

Case	CSA				CSA-R				CSA-I				CSA-IR				ICSA	
	C (A: ICSA; B: CSA)				C (A: ICSA; B: CSA-R)				C (A: ICSA; B: CSA-I)				C (A: ICSA; B: CSA-IR)					
	Avg IGD	Avg HVR	C(A,B)	C(B,A)	Avg IGD	Avg HVR	C(A,B)	C(B,A)	Avg IGD	Avg HVR	C(A,B)	C(B,A)	Avg IGD	Avg HVR	C(A,B)	C(B,A)	Avg IGD	Avg HVR
B1	0.525	0.100	0.875	0.125	0.526	0.219	0.923	0	0.104	0.769	0.738	0.250	0.300	0.485	0.844	0	0.114	0.835
B2	0.555	0.113	1	0	0.543	0.165	0.875	0	0.262	0.448	0.750	0.208	0.191	0.687	0.500	0.387	0.070	0.823
B3	0.599	0.158	0.891	0	0.607	0.108	0.875	0	0.192	0.629	0.500	0.250	0.275	0.529	0.500	0.250	0.132	0.804
B4	0.634	0.198	1	0	0.868	0.041	1	0	0.147	0.757	0.750	0.125	0.166	0.778	0.625	0.375	0.022	0.957
B5	0.381	0.239	0.875	0.125	0.377	0.196	1	0	0.155	0.600	0.583	0.204	0.233	0.456	0.750	0.250	0.073	0.832
B6	0.432	0.115	1	0	0.350	0.180	1	0	0.128	0.603	0.781	0.188	0.102	0.669	0.850	0.125	0.010	0.957
B7	0.667	0.159	0.781	0	0.606	0.112	1	0	0.277	0.546	0.875	0.125	0.189	0.662	0.625	0.146	0.039	0.962
B8	0.502	0.095	1	0	0.486	0.148	0.750	0	0.147	0.692	0.625	0.325	0.130	0.677	0.625	0.200	0.054	0.836
B9	0.448	0.051	1	0	0.366	0.168	1	0	0.139	0.632	0.708	0.250	0.172	0.507	0.625	0.188	0.024	0.917
B10	0.621	0.097	1	0	0.540	0.220	0.875	0	0.101	0.814	0.625	0.250	0.062	0.892	0.750	0.083	0.022	0.942
B11	0.731	0.044	1	0	0.605	0.148	1.000	0	0.074	0.790	0.500	0.382	0.104	0.774	0.500	0.396	0.090	0.892
B12	0.605	0.224	0.625	0	0.939	0.059	0.625	0	0.160	0.771	0.375	0.429	0.135	0.747	0.625	0.000	0.207	0.776
B13	0.401	0.160	0.775	0	0.307	0.248	0.875	0	0.149	0.583	0.875	0.125	0.203	0.489	0.625	0.125	0.053	0.858
B14	0.512	0.198	0.875	0	0.531	0.208	0.875	0.125	0.234	0.569	0.621	0.175	0.203	0.673	0.375	0.375	0.082	0.855
B15	0.587	0.171	0.875	0	0.566	0.178	0.875	0.125	0.240	0.626	0.750	0.250	0.227	0.562	0.750	0.063	0.190	0.803
B16	0.852	0.032	0.875	0	0.428	0.390	0.750	0.125	0.307	0.557	0.750	0.250	0.403	0.366	0.750	0.125	0.049	0.852
B17	0.533	0.068	0.875	0	0.437	0.168	0.750	0	0.228	0.533	0.625	0.283	0.229	0.500	0.650	0.250	0.116	0.743
B18	0.378	0.129	0.825	0	0.462	0.035	0.875	0	0.112	0.702	0.500	0.268	0.168	0.569	0.500	0.037	0.102	0.771
B19	0.722	0.020	1	0	0.453	0.182	0.750	0	0.180	0.634	0.500	0.375	0.178	0.585	0.625	0.250	0.165	0.771
B20	0.662	0.070	1	0	0.506	0.233	0.875	0	0.126	0.741	0.625	0.188	0.199	0.649	0.563	0.250	0.065	0.894
B21	0.458	0.274	0.875	0	0.450	0.224	0.875	0.125	0.149	0.640	0.750	0.250	0.268	0.485	0.750	0.179	0.078	0.829
B22	0.466	0.136	0.750	0.125	0.435	0.148	1	0	0.102	0.715	0.417	0.444	0.111	0.762	0.319	0.583	0.095	0.758
B23	0.599	0.079	1	0	0.382	0.246	0.875	0	0.117	0.787	0.625	0.150	0.158	0.599	0.750	0.125	0.016	0.951
B24	0.844	0.015	1	0	0.655	0.097	1.000	0	0.127	0.799	0.750	0.250	0.199	0.690	0.750	0.071	0.039	0.910
B25	0.469	0.138	0.958	0	0.442	0.188	0.750	0.073	0.203	0.515	0.625	0.250	0.375	0.289	0.750	0.010	0.107	0.841
B26	0.576	0.023	1	0	0.419	0.154	1	0	0.066	0.860	0.602	0.375	0.206	0.513	0.750	0.018	0.060	0.861
B27	0.401	0.127	1	0	0.412	0.120	1	0	0.147	0.594	0.643	0.175	0.107	0.728	0.481	0.132	0.034	0.910
B28	0.594	0.089	0.938	0	0.492	0.189	0.875	0.125	0.165	0.684	0.625	0.375	0.205	0.594	0.625	0.167	0.049	0.864
B29	0.465	0.126	1	0	0.469	0.105	1	0	0.243	0.511	0.625	0.125	0.289	0.460	0.750	0.125	0.083	0.863
B30	0.664	0.040	1	0	0.584	0.099	1	0	0.136	0.698	0.625	0.234	0.153	0.650	0.688	0.250	0.050	0.905
B31	0.736	0.027	1	0	0.573	0.145	0.792	0	0.301	0.493	0.500	0.250	0.177	0.663	0.563	0.375	0.167	0.703
B32	0.620	0.090	1	0	0.636	0.097	1	0	0.123	0.781	0.375	0.375	0.151	0.684	0.625	0.313	0.078	0.834

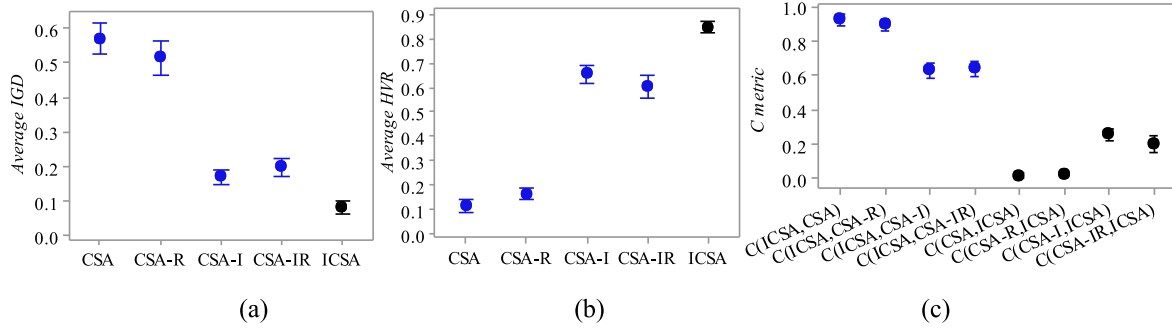


Fig. 6. Interval plot of the variants of ICSA: (a) Average IGD; (b) Average HVR; (c) C metric.

this sub-problem to cover more solution space. (2) The processing sub-problem is the foundation of other sub-problems, and its optimization has a great impact on the performance of the final scheduling result.

In sum, by adaptively selecting the sub-problems to be executed, the utilization rate of computing resources is improved and the exploration efficiency of each sub-problem is maximized. In the following experiments, we select the recommended actions with a probability of 0.9 and the other action with a probability of 0.1 to avoid overfitting.

5.3. Effectiveness of the proposed improvements

The proposed algorithm includes two key improvements: rule-based initialization and adaptive execution strategy of sub-problems. To evaluate the effectiveness of the two improvements, five variants are programmed and denoted as CSA, CSA-R, CSA-I, CSA-IR, and ICSA. Among them, CSA adopts random initialization and executes three sub-problems in sequence in each iteration. Different from CSA, CSA-R randomly and iteratively selects one or more sub-problems to be executed. CSA-I and CSA-IR respectively replace the random initialization of CSA and CSA-R with rule-based initialization. As for ICSA, it simultaneously applies rule-based initialization and adaptive execution strategy of sub-problems. The above five algorithms are tested on cases B1–B32. Under the same stopping criterion, each algorithm runs on each case 30 times to obtain the average IGD, HVR, and C metric. Table 5 and Fig. 6 show the comparison results. The best results are shown in bold.

It can be seen from Table 5 and Fig. 6 that in all cases, the IGD values of CSA-I are significantly smaller than those of CSA, while the HVR values are significantly larger than those of CSA. Since the smaller IGD and larger HVR mean that the algorithm has better convergence and diversity, it can be concluded that the performance of CSA-I is better than that of CSA, and the rule-based initialization is effective.

By comparing CSA and CSA-R, one can find that in both the IGD and HVR metrics, CSA-R performs better than CSA. But, the opposite result occurs when comparing CSA-I and CSA-IR. These results indicate that the execution strategy of sub-problems has a certain influence on the performance of coevolutionary algorithms. However, randomly

selecting the sub-problems to be executed may cause improper allocation of computing resources to each sub-problem, so as to cause the poor performance of CSA-IR. In contrast, by adopting an adaptive execution strategy of sub-problems, the ICSA significantly outperforms the CSA, CSA-I, CSA-R, and CSA-IR. This indicates that the adaptive execution strategy learned by Q-learning can effectively switch off/on the sub-problems to be executed according to the evolution state of each sub-problem, so as to maximize the overall benefits under limited computational resources.

In addition, in terms of the C metric, C (ICSA, the comparison one) is much bigger than C (the comparison one, ICSA) in almost all cases, which indicates that the Pareto frontier obtained by the proposed ICSA is closest to the true Pareto frontier, and can dominate most or even all of the solutions by the comparison algorithms. In conclusion, both the two proposed improvements are effective, and the combination of the two improvements can greatly improve the performance of ICSA.

5.4. Comparison with other algorithms

To further verify the performance of the improved coevolution simulated annealing algorithm (ICSA) in solving the studied multi-objective MAJSP-LS-BT, the ICSA is compared with four prevailing algorithms designed for solving similar problems. They are multi-population memetic algorithm with Q-learning (MPMA-QL) designed for a three-stage assembly hybrid flow shop scheduling problem (Jia et al., 2023), teaching-learning-based optimization (TLBO) for the assembly job shop scheduling with batch splitting (Sun et al., 2023), simulated annealing (SA) for the mixed-model assembly job shop scheduling problem (Cheng et al., 2022), and tabu search (TS) for the flexible job shop with lot streaming (Božek and Werner, 2018). In addition, a coevolution artificial bee colony algorithm (CABC) (Pan, 2016) and two classical multi-objective algorithms NSGA-II (Wang and Lu, 2021) and multi-objective imperialist competitive algorithm (MOICA) (Enayatifar et al., 2013) are also modified to solve the studied problem. Note that, to deal with the studied multi-objective problem, SA, TS, and CABC are respectively embedded with the Pareto

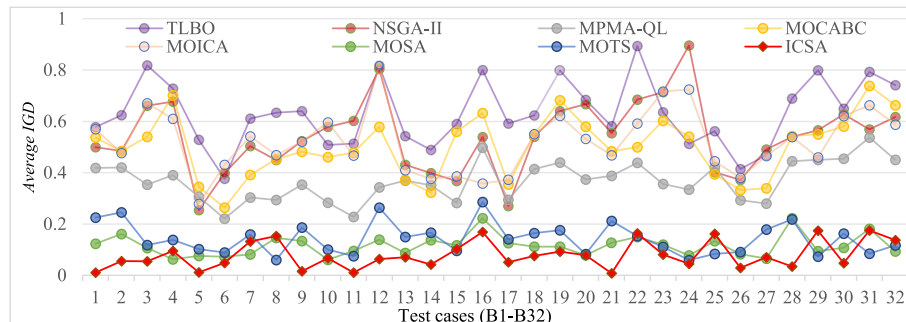


Fig. 7. Comparison results of ICSA with other algorithms in terms of average IGD.

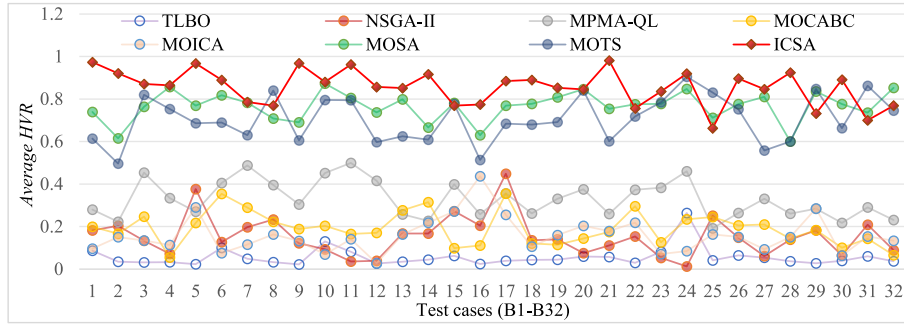


Fig. 8. Comparison results of ICSA with other algorithms in terms of average HVR.

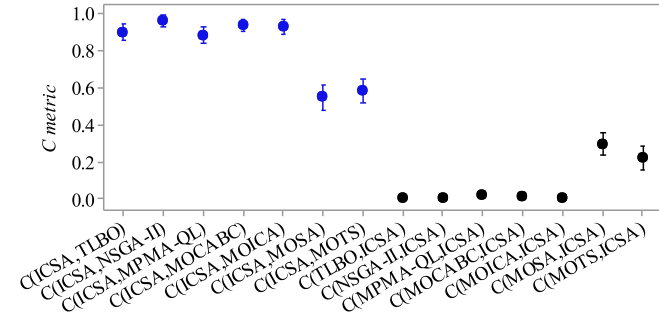


Fig. 9. Comparison results of ICSA with other algorithms in terms of average *C metric*.

non-dominant sorting method in NSGA-II, and the modified algorithms are denoted as MOSA, MOTS, and MOCABC.

All algorithms use the same encoding and decoding. In addition, MOSA and MOTS adopt the proposed rule-based initialization and neighborhood operators. MOCABC creates one overall solution via rule-based initialization and creates sub-swarms via random initialization. As for NSGA-II, MOICA, MPMA-QL, and TLBO, random initialization is adopted to create a population. All algorithms are tested on cases B1–B32 under the same stopping criterion. Also, each algorithm runs on each case 30 times, and the average *IGD*, *HVR*, *C metric* of 30 times are calculated and used as performance metrics.

Fig. 7, Fig. 8 and Fig. 9 show the comparison results. It can be seen that in all cases, the proposed ICSA performs far better than TLBO, NSGA-II, MPMA-QL, MOCABC and MOICA in terms of average *IGD* and *HVR*. In addition, except for a few cases, ICSA performs slightly worse than MOSA and MOTS, and in other cases, ICSA achieves the minimum *IGD* and maximum *HVR*. Regarding the *C metric*, *C*(ICSA, the comparison one) is much bigger than *C*(the comparison one, ICSA). This indicates that a big part of the Pareto frontier solution achieved by the comparison algorithms is dominated by solutions from ICSA, while the Pareto frontier solution by the comparison algorithms hardly Pareto dominates those by ICSA. In sum, the evaluation results in terms of *IGD*, *HVR* and *C metric* all show that compared with other comparison algorithms, the proposed ICSA can obtain Pareto frontier solutions with good distribution uniformity, and these solutions are closer to the real Pareto frontier.

Further, to investigate the statistical significance of the above results, the non-parametric Friedman test (Derrac et al., 2011) is conducted.

Table 6 presents the Friedman test results. It can be seen that ICSA achieves the minimum rank in *IGD* and the maximum rank in *HVR* with a *p*-value much less than 0.001. Thus, the proposed ICSA outperforms the other seven prevailing multi-objective algorithms with statistical significance.

5.5. Scenario analysis

To analyze the impact of lot streaming and batch transfer on mixed-model assembly job shop scheduling, four production scenarios are designed and compared. Scenario 1 is “processing without lot streaming-batch transfer-mixed-model assembly”. In this scenario, lot split is not considered and the processing unit is a lot. Scenario 2 is “processing with lot streaming-whole set transfer-mixed-model assembly”. In the processing stage, each lot is split into small sublots, and in the transfer stage, all components of a product are transferred together. Scenario 3 is “processing with lot streaming-batch transfer-mixed-model assembly”. Different from scenario 2, components from different products are allowed to be assigned to a batch and transferred together. Scenario 4 is “processing with lot streaming-batch transfer-mixed-model assembly considering pull production”. On the basis of scenario 3, pull production is considered to eliminate overproduction.

An illustrative case is provided, and the case under different production scenarios is solved by the proposed ICSA. The illustrative case contains two types of products with each of them having five components, and the lot size of the two products is ten and twelve. All components are processed on seven machines and then transferred and assembled on one mixed-model assembly line with three stations. Note that, for the first three production scenarios, only the semi-active decoding is adopted. As for scenario 4, two-stage decoding is applied to achieve pull production. Fig. 10 shows the Gantt charts of different production scenarios. In the processing stage, the same color indicates that the component comes from a subplot or lot, and the white squares represent setups. In the assembly stage, the same color represents the product from one type.

As revealed in Fig. 10 and Table 7, scenario 1 has the longest total completion time and total inventory time, but the setup times and the span time of assembly are the shortest. The reason is that without lot streaming, each component can start only after all components in a lot are ready, which incurs more waiting time and inventory time and greatly slows down the flow of production. In this situation, assembly operations cannot start early, and assembly completion time is extended. But, since the components from a lot are finished at a close time, the span time of assembly is shortened.

Table 6

Friedman test results.

Friedman test	TLBO	NSGA-II	MPMA-QL	MOCABC	MOICA	MOSA	MOTS	ICSA	<i>p</i> -value
Rank of <i>IGD</i>	7.63	6.25	4.22	5.75	6.16	1.97	2.5	1.53	1.83E-38
Rank of <i>HVR</i>	1.28	2.81	4.72	3.34	2.84	6.97	6.41	7.63	6.55E-39

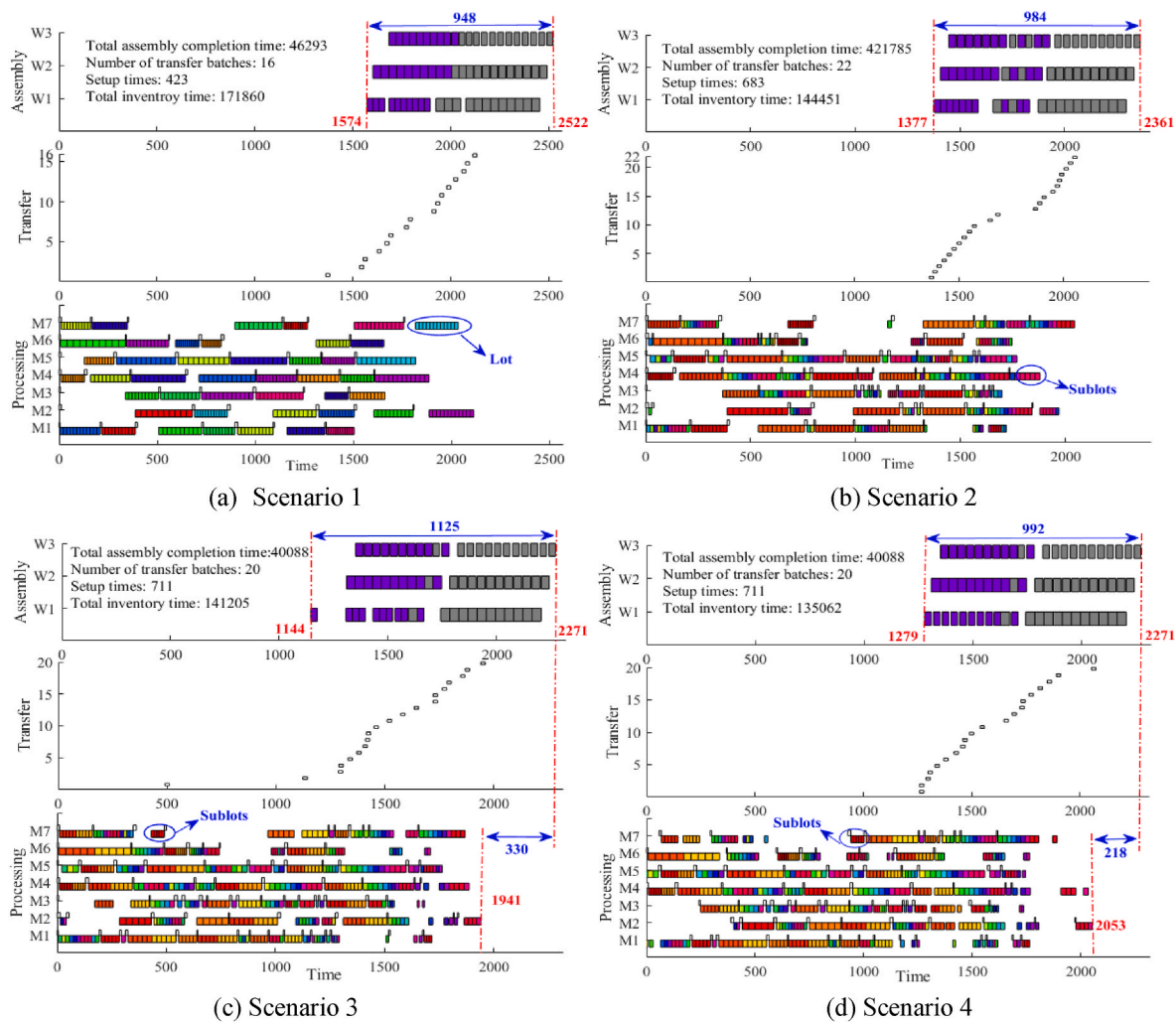


Fig. 10. Gantt charts of four production scenarios.

Table 7
Comparison results of schedule performance in four production scenarios.

Performance index	Scenario 1	Scenario 2	Scenario 3	Scenario 4
Total completion time	40088	421785	40088	40088
Number of transfer batches	20	22	20	20
Setup times	423	683	711	711
Total inventory time	141205	144451	141205	135062
Finish time of processing	2114	2043	1941	2053
Start time of assembly	1574	1377	1144	1279
Finish time of assembly	2522	2361	2271	2271
Span time of assembly	948	984	1125	992

In scenarios 2 and 3, due to the introduction of lot streaming, setup times increase a bit but components can be finished early and then sent to assembly earlier. As a result, the total assembly completion time is greatly shortened. Especially, restricted by the whole set transfer in scenario 2, components from different products cannot be transferred together even if they have a close finish time or the load capacity of the transport device is still surplus. In this situation, more transfer batches may be required on the one hand, on the other hand, some components finished earlier cannot be sent to assembly in time, resulting in increased inventory time and longer assembly completion time. In contrast, scenario 3 has a more flexible batch transfer approach, which allows components to be transferred in the right batch and at the right time, resulting in lower inventory, fewer transfer batches, and shorter assembly completion time.

Scenario 4 is an enhanced version of scenario 3. It can be seen that as opposed to processing components as early as possible in scenario 3, pull production tries to ensure that components are processed when they are needed at each assembly station. In this case, the difference between the finish time of processing and assembly is reduced, and a lower inventory time is obtained without deterioration of other performance criteria.

In sum, the above results reveal that the developed production mode of “processing with lot streaming-batch transfer-mixed-model assembly considering pull production” requires a shorter completion time, fewer setups, fewer transfer batches and less WIP inventory, and it is more conducive to the coordination and synchronization of production and logistics in discrete manufacturing enterprises.

6. Conclusion and future work

This paper pays attention to a three-stage mixed-model assembly job shop scheduling considering lot streaming in the processing stage, mixed-model assembly in the assembly stage, and batch transfer of components between the processing and assembly stages. To tackle this problem, a MILP model and an improved multi-objective coevolutionary simulated annealing algorithm are developed to minimize total completion time and production costs including setup costs, transport costs, and inventory costs. In the MILP model, lot streaming constraints, setup constraints, batch transfer constraints, mixed-model assembly constraints and spatial-temporal links among three stages with differentiated production lot size are considered. Two sets of linearization

methods are proposed to make the model linear. Regarding the proposed algorithm, a two-stage decoding is designed according to the characteristics of optimization objectives to find a better scheduling scheme. Three rules including the multi-attribute combination rule, similarity rule, and consistency rule are designed to create a high-quality initial solution for each sub-problem. An adaptive execution strategy is trained by Q-learning to adaptively switch off/on the sub-problems to be executed according to the evolution state of each sub-problem, so as to maximize the overall benefits.

From the experiment results, we found that both the discovered rule and the adaptive strategy are effective, and the proposed algorithm significantly outperforms seven competitive multi-objective algorithms in fixing the studied problem. More importantly, compared with the other three actual production modes, the developed production mode of “processing with lot streaming-batch transfer-mixed-model assembly considering pull production” has a better comprehensive performance in terms of completion time, setups, number of transfer batches, and WIP inventory.

For future work, more generalized situations where the number of transport devices is infinite are worth exploring, and task allocation of transport devices needs to be considered. Besides, adaptive selection mechanisms of key parameters and neighborhood operators can be designed to enhance the performance of algorithms.

CRedit authorship contribution statement

Lixin Cheng: Writing – original draft, Methodology, Software, Validation. **Qiuhua Tang:** Writing – review & editing, Supervision. **Liping Zhang:** Writing-Review, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgment

This work is supported by the National Natural Science Foundation of China (No. 52275504).

References

- Al-Anzi, F.S., Allahverdi, A., 2013. An artificial immune system heuristic for two-stage multi-machine assembly scheduling problem to minimize total completion time. *J. Manuf. Syst.* 32, 825–830. <https://doi.org/10.1016/j.jmsy.2013.06.001>.
- Alfieri, A., Garraffa, M., Pastore, E., et al., 2023. Permutation flowshop problems minimizing core waiting time and core idle time. *Comput. Ind. Eng.* 176, 108983 <https://doi.org/10.1016/j.cie.2023.108983>.
- Basir, S.A., Mazdeh, M.M., Namakshenas, M., 2018. Bi-level genetic algorithms for a two-stage assembly flow-shop scheduling problem with batch delivery system. *Comput. Ind. Eng.* 126, 217–231. <https://doi.org/10.1016/j.cie.2018.09.035>.
- Beren, G.Y., Ömer, F.Y., 2022. Lot streaming in hybrid flowshop scheduling problem by considering equal and consistent sublots under machine capability and limited waiting time constraint. *Comput. Ind. Eng.* 173, 108745 <https://doi.org/10.1016/j.cie.2022.108745>.
- Bozek, A., Werner, F., 2018. Flexible job shop scheduling with lot streaming and subplot size optimisation. *Int. J. Prod. Res.* 56, 6391–6411. <https://doi.org/10.1080/00207543.2017.1346322>.
- Chan, F.T.S., Wong, T.C., Chan, L.Y., 2008. Lot streaming for product assembly in job shop environment. *ROBOT CIM-INT MANUF* 24, 321–331. <https://doi.org/10.1016/j.rcim.2007.01.001>.
- Chang, X., Shi, J., Luo, Z., et al., 2023. Adaptive large neighborhood search algorithm for multi-stage weapon target assignment problem. *Comput. Ind. Eng.* 181 <https://doi.org/10.1016/j.cie.2023.109303>.
- Chen, S.Y., Wang, X.W., Wang, Y., et al., 2023. A modified adaptive switching-based many-objective evolutionary algorithm for distributed heterogeneous flowshop scheduling with lot-streaming. *Swarm Evol. Comput.* 81, 101353 <https://doi.org/10.1016/j.swevo.2023.101353>.
- Cheng, L., Tang, Q., Zhang, L., et al., 2022. Inventory and total completion time minimization for assembly job-shop scheduling considering material integrity and assembly sequential constraint. *J. Manuf. Syst.* 65, 660–672. <https://doi.org/10.1016/j.jmsy.2022.10.013>.
- Daneshamooz, F., Fattahi, P., Hosseini, S.M.H., 2022. Scheduling in a flexible job shop followed by some parallel assembly stations considering lot streaming. *Eng Optimiz* 54, 614–633. <https://doi.org/10.1080/0305215X.2021.1887168>.
- Derrac, J., Garcia, S., Molina, D., 2011. A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms. *Swarm Evol. Comput.* 1, 3–18. <https://doi.org/10.1016/j.swevo.2011.02.002>.
- Enayatifar, R., Yousefi, M., Abdullah, A.H., et al., 2013. MOICA: a novel multi-objective approach based on imperialist competitive algorithm. *Appl. Math. Comput.* 219, 8829–8841. <https://doi.org/10.1016/j.amc.2013.03.099>.
- Fattahi, P., Bagheri Rad, N., Daneshamooz, F., et al., 2020. A new hybrid particle swarm optimization and parallel variable neighborhood search algorithm for flexible job shop scheduling with assembly process. *Assembly Autom* 40, 419–432. <https://doi.org/10.1108/aa-11-2018-0178>.
- Fernandez-Viagas, V., Talens, C., Framinan, J.M., 2022. Assembly flowshop scheduling problem: speed-up procedure and computational evaluation. *Eur. J. Oper. Res.* 299, 869–882. <https://doi.org/10.1016/j.ejor.2021.10.001>.
- Gong, D.W., Han, Y.Y., Sun, J.Y., 2018. A novel hybrid multi-objective artificial bee colony algorithm for blocking lot-streaming flow shop scheduling problems. *Knowl-Based Syst* 148, 115–130. <https://doi.org/10.1016/j.knsys.2018.02.029>.
- Guo, G., Ryan, S.M., 2021. Sequencing mixed-model assembly lines with risk-averse stochastic mixed-integer programming. *Int. J. Prod. Res.* 60, 1–18. <https://doi.org/10.1080/00207543.2021.1931978>.
- Guo, W., Lei, Q., Song, Y., et al., 2021. A learning interactive genetic algorithm based on edge selection encoding for assembly job shop scheduling problem. *Comput. Ind. Eng.* 159 <https://doi.org/10.1016/j.cie.2021.107455>.
- Hajibabaei, M., Behnamian, J., 2023. Fuzzy cleaner production in assembly flexible job-shop scheduling with machine breakdown and batch transportation: Lagrangian relaxation. *J. Comb Optim* 45. <https://doi.org/10.1007/s10878-023-01046-1>.
- Hatami, S., Ebrahimnejad, S., Tavakkoli-Moghaddam, R., et al., 2010. Two meta-heuristics for three-stage assembly flowshop scheduling with sequence-dependent setup times. *Int J Adv Manuf Tech* 50, 1153–1164. <https://doi.org/10.1007/s00170-010-2579-5>.
- Jia, Y.H., Yan, Q., Wang, H.F., 2023. Q-learning driven multi-population memetic algorithm for distributed three-stage assembly hybrid flow shop scheduling with flexible preventive maintenance. *Expert Syst. Appl.* 232, 120837 <https://doi.org/10.1016/j.eswa.2023.120837>.
- Kazemi, H., Mazdeh Mohammad, M., Rostami, M., 2017. The two stage assembly flow-shop scheduling problem with batching and delivery. *Eng. Appl. Artif. Intell.* 63, 98–107. <https://doi.org/10.1016/j.engappai.2017.05.004>.
- Komaki, G.M., Kayvanfar, V., 2015. Grey Wolf Optimizer algorithm for the two-stage assembly flow shop scheduling problem with release time. *J. Comput Sci-Neth* 8, 109–120. <https://doi.org/10.1016/j.jocs.2015.03.011>.
- Lei, D.M., Wang, J., Cai, J.C., 2022. An adaptive artificial bee colony with reinforcement learning for distributed three-stage assembly scheduling with maintenance. *Appl. Soft Comput.* 117, 1–13. <https://doi.org/10.1016/j.asoc.2021.108371>.
- Li, X., Lu, J., Yang, C., et al., 2022. Research of flexible assembly job-shop batch-scheduling problem based on improved artificial bee colony. *Front Bioeng Biotech* 10, 909548. <https://doi.org/10.3389/fbioe.2022.909548>.
- Li, Z.X., Tang, Q.H., Zhang, L.P., 2016. Minimizing energy consumption and cycle time in two-sided robotic assembly line systems using restarted simulated annealing algorithm. *J. Clean. Prod.* 135, 508–522. <https://doi.org/10.1016/j.jclepro.2016.06.131>.
- Liu, Z., Guo, S., Wang, L., 2019. Integrated green scheduling optimization of flexible job shop and crane transportation considering comprehensive energy consumption. *J. Clean. Prod.* 211, 765–786. <https://doi.org/10.1016/j.jclepro.2018.11.231>.
- Meng, K., Tang, Q., Zhang, Z., 2022. Robust assembly line balancing problem considering preventive maintenance scenarios with interval processing time. *Eng. Appl. Artif. Intell.* 116 <https://doi.org/10.1016/j.engappai.2022.105417>.
- Novas, J.M., 2019. Production scheduling and lot streaming at flexible job-shops environments using constraint programming. *Comput. Ind. Eng.* 136, 252–264. <https://doi.org/10.1016/j.cie.2019.07.011>.
- Pan, Q.K., 2016. An effective co-evolutionary artificial bee colony algorithm for steelmaking-continuous casting scheduling. *Eur. J. Oper. Res.* 250, 702–714. <https://doi.org/10.1016/j.ejor.2015.10.007>.
- Pang, S., Guo, S., Wang, L., et al., 2021. Mass personalization-oriented integrated optimization of production task splitting and scheduling in a multi-stage flexible assembly shop. *Comput. Ind. Eng.* 162 <https://doi.org/10.1016/j.cie.2021.107736>.
- Potter, M.A., de Jong, K.A., 2000. Cooperative coevolution: an architecture for evolving coadapted subcomponents. *Evol. Comput.* 8, 1–29. <https://doi.org/10.1162/106365600568086>.
- Ren, W., Wen, J., Yan, Y., et al., 2021. Multi-objective optimisation for energy-aware flexible job-shop scheduling problem with assembly operations. *Int. J. Prod. Res.* 59, 7216–7231. <https://doi.org/10.1080/00207543.2020.1836421>.
- Sheikh, S., Komaki, G.M., Kayvanfar, V., et al., 2019. Multi-Stage assembly flow shop with setup time and release time. *Operations Research Perspectives* 6, 1–15. <https://doi.org/10.1016/j.orp.2019.100111>.
- Shoaeidehbil, N., Fattahi, P., 2015. Multi-objective meta-heuristics to solve three-stage assembly flow shop scheduling problem with machine availability constraints. *Int. J. Prod. Res.* 53, 944–968. <https://doi.org/10.1080/00207543.2014.948575>.

- Sinaga, K.P., Yang, M.-S., 2020. Unsupervised K-means clustering algorithm. *IEEE Access* 8, 80716–80727. <https://doi.org/10.1109/access.2020.2988796>.
- Sun, M.K., Cai, Z.Y., Zhang, H.N., 2023. A teaching-learning-based optimization with feedback for L-R fuzzy flexible assembly job shop scheduling problem with batch splitting. *Expert Syst. Appl.* 224, 120043 <https://doi.org/10.1016/j.eswa.2023.120043>.
- Tang, Q., Zhang, Z., 2021. Integrating flexible preventive maintenance activities into two-stage assembly flow shop scheduling with multiple assembly machines. *Comput. Ind. Eng.* 159 <https://doi.org/10.1016/j.cie.2021.107493>.
- Tang, Q., Zhang, Z., Chica, M., 2021. A robust MILP and gene expression programming based on heuristic rules for mixed-model multi-manned assembly line balancing. *Appl. Soft Comput.* 109, 1–20. <https://doi.org/10.1016/j.asoc.2021.107513>.
- Tutumlu, B., Saraç, T., 2023. A MIP model and a hybrid genetic algorithm for flexible job-shop scheduling problem with job-splitting. *Comput. Oper. Res.* 155 <https://doi.org/10.1016/j.cor.2023.106222>.
- Wang, H., Sarker, B.R., Li, J., et al., 2021. Adaptive scheduling for assembly job shop with uncertain assembly times based on dual Q-learning. *Int. J. Prod. Res.* 59, 5867–5883. <https://doi.org/10.1080/00207543.2020.1794075>.
- Wang, J., Tang, H.T., Lei, D.M., 2023. A Q-learning artificial bee colony for distributed assembly flow shop scheduling with factory eligibility, transportation capacity and setup time. *Eng. Appl. Artif. Intell.* 123, 106230 <https://doi.org/10.1016/j.engappai.2023.106230>.
- Wang, J.J., Wang, L., 2022. A cooperative memetic algorithm with feedback for the energy-aware distributed flow-shops with flexible assembly scheduling. *Comput. Ind. Eng.* 168 <https://doi.org/10.1016/j.cie.2022.108126>.
- Wang, K., Ma, W.Q., Luo, H., et al., 2016. Coordinated scheduling of production and transportation in a two-stage assembly flowshop. *Int. J. Prod. Res.* 54, 6891–6911. <https://doi.org/10.1080/00207543.2016.1193246>.
- Wang, W., Xu, Z., Gu, X., 2022. A two-stage discrete water wave optimization algorithm for the flowshop lot-streaming scheduling problem with intermingling and variable lot sizes. *Knowl-Based Syst.* 238. <https://doi.org/10.1016/j.knosys.2021.107874>.
- Wang, Z.Y., Lu, C., 2021. An integrated job shop scheduling and assembly sequence planning approach for discrete manufacturing. *J. Manuf. Syst.* 61, 27–44. <https://doi.org/10.1016/j.jmsy.2021.08.003>.
- Wong, T.C., Chan, F.T.S., Chan, L.Y., 2009. A resource-constrained assembly job shop scheduling problem with Lot Streaming technique. *Comput. Ind. Eng.* 57, 983–995. <https://doi.org/10.1016/j.cie.2009.04.002>.
- Wong, T.C., Ngan, S.C., 2013. A comparison of hybrid genetic algorithm and hybrid particle swarm optimization to minimize makespan for assembly job shop. *Appl. Soft Comput.* 13, 1391–1399. <https://doi.org/10.1016/j.asoc.2012.04.007>.
- Wu, X.L., Liu, X.J., Zhao, N., 2018. An improved differential evolution algorithm for solving a distributed assembly flexible job shop scheduling problem. *Memet Comput.* 11, 335–355. <https://doi.org/10.1007/s12293-018-00278-7>.
- Xie, F., Li, L., Li, L., et al., 2023. A decomposition-based multi-objective Jaya algorithm for lot-streaming job shop scheduling with variable sublots and intermingling setting. *Expert Syst. Appl.* 228 <https://doi.org/10.1016/j.eswa.2023.120402>.
- Yang, S., Xu, Z., 2021. The distributed assembly permutation flowshop scheduling problem with flexible assembly and batch delivery. *Int. J. Prod. Res.* 59, 4053–4071. <https://doi.org/10.1080/00207543.2020.1757174>.
- Yu, H., Gao, K.Z., Ma, Z.F., et al., 2023. Improved meta-heuristics with Q-learning for solving distributed assembly permutation flowshop scheduling problems. *Swarm Evol. Comput.* 80 <https://doi.org/10.1016/j.swevo.2023.101335>.
- Zhang, B., Pan, Q.K., Meng, L.L., et al., 2021. A collaborative variable neighborhood descent algorithm for the hybrid flowshop scheduling problem with consistent sublots. *Appl. Soft Comput.* 106 <https://doi.org/10.1016/j.asoc.2021.107305>.
- Zhang, S., Li, X., Zhang, B., et al., 2020. Multi-objective optimisation in flexible assembly job shop scheduling using a distributed ant colony system. *Eur. J. Oper. Res.* 283, 441–460. <https://doi.org/10.1016/j.ejor.2019.11.016>.
- Zhang, Z., Tang, Q., Chica, M., 2021. Maintenance costs and makespan minimization for assembly permutation flow shop scheduling by considering preventive and corrective maintenance. *J. Manuf. Syst.* 59, 549–564. <https://doi.org/10.1016/j.jmsy.2021.03.020>.
- Zhao, F., Zhao, J., Wang, L., et al., 2021. An optimal block knowledge driven backtracking search algorithm for distributed assembly No-wait flow shop scheduling problem. *Appl. Soft Comput.* 112 <https://doi.org/10.1016/j.asoc.2021.107750>.
- Zhao, F., Di, S., Wang, L., et al., 2022. A self-learning hyper-heuristic for the distributed assembly blocking flow shop scheduling problem with total flowtime criterion. *Eng. Appl. Artif. Intell.* 116 <https://doi.org/10.1016/j.engappai.2022.105418>.